System design patterns

06/08/2026 12:36:03
https://claude.ai/chat/e505d378-2d88-425f-8f3a-0aface3313b6

Write 1 page per topic: Load Balancer (mention types of load balancer as wel...

System Design Interview Notes --- ## 1. Load Balancer **What it is:** A loa...

webhooks basics Grpc Basics exception handling patterns performance tuni...

System Design — Round 2 Notes --- ## 1. Webhooks Basics **What it is:** ...

How to do db partition what are the ways

Database Partitioning — Complete Guide --- ## What is Partitioning?...

more system design Q&A for a senior engineer and principal engineer

System Design Q&A — Senior & Principal Engineer Level --- ## Category 1:...

is this enough

Honestly — for a **Senior Engineer** level: yes, more than enough. For...

give more Q&A for principal engineer

Principal Engineer System Design Q&A --- ## Category 1: Architecture...

is this much is enough

Honest answer — **yes, for Senior. Almost for Principal.** Here's the real...

is it cover ? TopicWhy It MattersData modeling for scaleHow you design...

Not yet. Let me cover all 5 now. --- # Principal Engineer — The 5 Missing Topi...

Write 1 page per topic:
Load Balancer (mention types of load balancer as well)
Database (SQL vs NoSQL)
Cache (Redis basics) (mention types of cache as well)
Message Queue (Kafka / RabbitMQ idea)
API Gateway
Microservices (Monolithic vs Microservices)
👉 For each:
what it is
why we use it
simple example

“How does caching work?”

“Why use message queue?”

"and when there is kafka for event processing again why we should impliment any QUeue here whats the need"

“SQL vs NoSQL difference?”

“How to scale system?”

👉 Keep answers 5–10 lines max

explain these 2

Redis caching strategies

Consistency models

Scalable system design basics

API design best practices

Distributed systems basics

6/5/2026, 10:21:03 AM

System Design Interview Notes

1. Load Balancer

What it is: A load balancer sits between clients and your backend servers. It receives incoming requests and distributes them across multiple server instances so no single server gets overwhelmed.

Why we use it: Without a load balancer, one server handles all traffic — it becomes a bottleneck and a single point of failure. With it, you get high availability, horizontal scalability, and fault tolerance. If one server goes down, the load balancer stops sending traffic to it automatically.

Simple example: You have 3 instances of your Spring Boot Claims service behind an AWS Application Load Balancer. 1000 requests per second arrive — ALB routes ~333 to each instance using round-robin. If instance 2 crashes, ALB detects it via health checks and splits traffic between instances 1 and 3.

Types of Load Balancers:

- **Layer 4 (Transport layer):** Routes based on IP and TCP/UDP port. Very fast, no content inspection. AWS Network Load Balancer (NLB) works at this level. Use when you need ultra-low latency (gaming, financial tick data).
- **Layer 7 (Application layer):** Routes based on HTTP headers, URL paths, cookies, query params. AWS ALB works here. Use when you need path-based routing (`/claims` → service A, `/auth` → service B), host-based routing, or SSL termination.
- **Hardware load balancers:** Physical appliances (F5, Citrix). Expensive, high performance, used in on-prem enterprise setups.
- **Software load balancers:** Nginx, HAProxy, Envoy. Flexible, cheap, widely used in cloud/k8s setups.
- **DNS load balancing:** Multiple IPs returned for the same domain. Simple but no real-time health awareness.
- **Client-side load balancing:** The client holds the list of servers and picks one (used in service meshes, Spring Cloud Ribbon). No central bottleneck.

Load balancing algorithms:

- Round-robin — requests distributed in order
- Least connections — send to server with fewest active connections

- IP hash — same client always hits the same server (useful for session stickiness)
- Weighted round-robin — heavier servers get more traffic

---

2. Database (SQL vs NoSQL)

What it is: A database is a structured system for persisting and querying data. SQL databases store data in tables with a fixed schema and relationships. NoSQL databases store data in flexible formats — documents, key-value pairs, wide columns, or graphs.

Why we use it: Every application needs persistent storage. The choice between SQL and NoSQL depends on your data shape, query patterns, consistency needs, and scale requirements.

Simple example: In Availity's Claims module — claim metadata (claimId, patientId, status, amount) lives in Aurora MySQL (SQL) because it needs ACID transactions and joins. The claim document payload (full ANSI 837 data, variable fields) lives in MongoDB (NoSQL) because its shape varies by payer and doesn't fit neatly into columns — this is exactly the CQRS pattern you use.

SQL vs NoSQL — Side by Side:

Aspect	SQL	NoSQL
Schema	Fixed, predefined	Flexible, dynamic
Data model	Tables, rows, columns	Document, key-value, wide-column, graph
Relationships	Foreign keys, JOINs	Embedded documents or application-level joins
Transactions	Full ACID	Eventual consistency (usually); some support ACID
Scaling	Vertical (scale up), limited horizontal	Horizontal (scale out) by design
Query language	Standardized SQL	Database-specific APIs
Best for	Financial records, user accounts, orders	Catalogs, event logs, user sessions, social graphs
Examples	MySQL, PostgreSQL, Aurora	MongoDB, Cassandra, DynamoDB, Redis

When to choose SQL: You have relational data, need multi-table transactions, your schema is stable, and correctness > flexibility (billing, claims, payroll).

When to choose NoSQL: Schema evolves frequently, you need massive horizontal scale, data is hierarchical or unstructured, and you can tolerate eventual consistency (product catalogs, activity feeds, sessions).

---

3. Cache (Redis Basics)

What it is: A cache is a fast in-memory store that holds frequently accessed data so your application doesn't have to hit the database every time. Redis is the most widely used cache — it stores data as key-value pairs in RAM, supports rich data structures (strings, hashes, lists, sets, sorted sets), and can persist data to disk.

Why we use it: Database reads are slow (10–100ms). Redis reads are fast (~0.1ms). For data that doesn't change on every request — user sessions, config, lookup tables, API responses — caching eliminates redundant DB calls, reduces latency, and protects your DB from being overwhelmed.

Simple example: Your Claims service fetches payer configuration on every claim submission. That config rarely changes. Without cache: 1000 claims/sec = 1000 DB reads/sec. With Redis: first request reads from DB and writes to Redis with a 5-minute TTL. Next 999 requests read from Redis — DB load drops to near zero.

```
// Spring Boot example
@Cacheable(value = "payerConfig", key = "#payerId")
public PayerConfig getPayerConfig(String payerId) {
    return payerConfigRepository.findById(payerId).orElseThrow();
}
```

Redis Data Structures:

- `String` — simple values, counters (`SET claim:123:status "SUBMITTED"`)
- `Hash` — object fields (`HSET user:42 name "Thabira" role "SDE"`)
- `List` — ordered sequences, queues
- `Set` — unique values, membership checks
- `Sorted Set` — leaderboard-style ranking with scores

- `TTL` — every key can have an expiry time

Types of Cache:

- **Application cache (in-process):** Cache lives inside the application's memory (e.g., Guava Cache, Caffeine). Ultra-fast, no network hop. Problem: each instance has its own copy — inconsistency across instances.
- **Distributed cache:** Cache lives in a separate shared service (Redis, Memcached). All app instances read from the same cache. Consistent, scalable. Small network latency vs in-process but worth it.
- **CDN cache:** Static assets (images, JS, CSS) cached at edge nodes (CloudFront). Geographically close to users.
- **Database query cache:** Some DBs cache query results internally (MySQL query cache — now deprecated). Limited control.
- **Write-through cache:** Write goes to cache AND DB at the same time. Cache always consistent. Slightly slower writes.
- **Write-back (write-behind) cache:** Write goes to cache first, DB updated asynchronously. Faster writes, risk of data loss if cache crashes.
- **Read-through cache:** Application always reads from cache; cache fetches from DB on miss and populates itself. Transparent to the app.
- **Cache-aside (lazy loading):** App checks cache first. On miss, app reads from DB and populates cache. Most common pattern. You own the logic.

Cache eviction policies (what Redis removes when full):

- `LRU` (Least Recently Used) — evict the key not accessed for the longest time
- `LFU` (Least Frequently Used) — evict the key accessed the fewest times
- `TTL-based` — keys expire on their own schedule
- `No eviction` — Redis returns error when full (good for session stores where you never want silent data loss)

---

4. Message Queue (Kafka / RabbitMQ)

What it is: A message queue is a system that lets services communicate asynchronously. Producer services write messages to the queue. Consumer services read from it at their own pace. Neither side needs to be online at the same time.

Why we use it: Without a queue, Service A calls Service B directly (synchronous). If B is slow, A is slow. If B is down, A fails. A queue decouples them — A writes to the queue and moves on. B processes when ready. You also get natural buffering (queue absorbs traffic spikes), retry capability, and fan-out (multiple consumers reading the same message).

Simple example: Claim is submitted → Claims service writes a `ClaimSubmitted` event to Kafka → three consumers read it independently: Eligibility service validates coverage, Audit service logs it, Notification service emails the provider. Claims service doesn't wait for any of them.

Kafka vs RabbitMQ:

Aspect	Kafka	RabbitMQ
Model	Log-based, append-only, pull	Message broker, push to consumers
Message retention	Retained for configurable period (days/weeks)	Deleted after acknowledgment
Throughput	Millions of messages/sec	Hundreds of thousands/sec
Consumer model	Consumer group reads at its own offset	Queue-based, message gone once consumed
Replay	Yes — rewind offset to replay old messages	No — consumed = gone
Ordering	Per partition	Per queue
Best for	Event streaming, audit logs, data pipelines, event sourcing	Task queues, RPC, work distribution, routing logic
Use in your stack	Kafka is already in use at Availity	RabbitMQ common in simpler microservice setups

When to use Kafka: You need event streaming, replay, high throughput, audit trails, or multiple independent consumers on the same event.

When to use RabbitMQ: You need task queues with complex routing (topic/fanout/direct exchanges), priority queues, or simpler pub-sub with lower operational overhead.

---

5. API Gateway

What it is: An API Gateway is a single entry point that sits in front of all your microservices. Every external client request passes through it. It handles routing to the right service, authentication, rate limiting, SSL termination, request/response transformation, logging, and more.

Why we use it: Without a gateway, clients need to know the address of every microservice, every service implements its own auth and rate limiting, and cross-cutting concerns get duplicated everywhere. The gateway centralizes all of that. It also protects your internal services from being exposed directly.

Simple example: A frontend React app calls `/api/claims/submit`. It doesn't know or care that this hits the Claims microservice at `http://claims-svc:8080/internal/claims`. The gateway (Tyk, in your case) receives the request, validates the JWT token, checks rate limits, transforms the path, and proxies to the correct service. The frontend sees one clean API.

What an API Gateway does:

- **Routing** — maps external paths to internal service URLs
- **Authentication / Authorization** — validate JWT, OAuth tokens before the request reaches services
- **Rate limiting** — prevent abuse (100 req/min per API key)
- **SSL termination** — HTTPS ends at the gateway; internal traffic can be HTTP
- **Load balancing** — distribute across service instances
- **Request/response transformation** — add headers, rewrite paths, reshape payloads
- **Logging and tracing** — centralized access logs, correlation IDs injected here
- **Circuit breaking** — stop forwarding to unhealthy services

Examples: AWS API Gateway, Kong, Tyk (your stack), Nginx, Apigee, Envoy.

---

6. Microservices (Monolithic vs Microservices)

What it is: A monolith is a single deployable unit where all modules (claims, auth, billing, notifications) live in one codebase and are deployed together. Microservices splits the application into small, independently deployable services, each owning its own data and deployed separately.

Why we use it: Monoliths are simple to start with but become painful at scale — one team's change requires redeploying everything, a bug in billing can crash claims, and you can't scale just the busy parts. Microservices let teams own and deploy their services independently, scale only what's needed, and use different tech stacks per service.

Simple example: Availity's Essential Pro is a microservices system. The Claims service, Eligibility service, Auth service, and Notification service are all separate Spring Boot apps deployed as separate containers in Kubernetes. A code change to Claims triggers only the Claims CI/CD pipeline — nothing else restarts.

Monolith vs Microservices:

Aspect	Monolith	Microservices
Deployment	Single unit — all or nothing	Each service deploys independently
Scaling	Scale everything even if only one module is busy	Scale individual services
Development	Simple locally, fast to start	Complex setup, need service discovery, API gateway, etc.
Failure isolation	One bug can bring down the whole app	Failure isolated to one service
Data	Shared database	Each service owns its own database
Team structure	One team owns it all	Each team owns a service
Latency	In-process calls (fast)	Network calls between services (adds latency)
Best for	Early-stage products, small teams	Mature products, large teams, high scale

Microservices challenges to be aware of in interviews:

- Distributed tracing needed (Zipkin, Jaeger, AWS X-Ray)

- Data consistency across services requires Sagas or event-driven patterns
- Network failures need circuit breakers (Resilience4j)
- Service discovery needed (Kubernetes DNS, Eureka)
- Debugging is harder — logs are spread across services

Quick-Fire Questions (5–10 lines each)

How does caching work?

Your application first checks the cache (Redis) for the data. If found (cache hit), it returns immediately — no DB call. If not found (cache miss), it reads from the database, stores the result in Redis with a TTL, and returns. On subsequent requests within the TTL window, the cache always hits. The key design decisions are: what to cache (hot data, expensive queries), how long to cache it (TTL), and how to invalidate it when the source changes. Caching works best for read-heavy data that doesn't change often — config tables, reference data, user sessions, API responses.

Why use a message queue?

Message queues decouple the producer from the consumer. The producer writes to the queue and immediately moves on — it doesn't wait for the consumer to process. This means: slow consumers don't block producers, traffic spikes are absorbed by the queue buffer, consumers can retry on failure without the producer knowing, and multiple consumers can process the same message independently. Without a queue, you're doing synchronous REST calls between services — if the downstream service is slow or down, your whole flow breaks.

When there's Kafka for event processing, why implement a queue? What's the need?

Kafka and traditional queues (RabbitMQ, SQS) are complementary, not interchangeable. Kafka is a distributed event log — it's brilliant for high-throughput event streaming, replay, audit logs, and fan-out to many consumers. But Kafka is overkill for simple task distribution. If you have a background job — send an email, resize an image, process a payment — a simple queue like SQS or RabbitMQ is lighter, easier, and has built-in dead-letter queues and visibility timeouts designed for task semantics. Also, Kafka requires

offsets, consumer groups, and partition management — for a job queue with 10 messages/sec, that's unnecessary complexity. Use Kafka when you need streaming and replay. Use a queue when you need reliable task dispatch.

SQL vs NoSQL difference?

SQL databases store data in structured tables with a fixed schema and enforce relationships via foreign keys. They support ACID transactions — critical when correctness matters (banking, claims). NoSQL databases use flexible formats (documents, key-value, wide-column) with no rigid schema — great when data shape varies or evolves. SQL scales vertically (bigger machine) and has limited horizontal sharding. NoSQL scales horizontally by design. Use SQL when your data is relational and transactions span multiple entities. Use NoSQL when you need schema flexibility, massive write throughput, or hierarchical data that doesn't fit rows and columns cleanly.

How to scale a system?

Start vertical (bigger machine) but that has a ceiling. Real scaling is horizontal — add more instances behind a load balancer. Cache aggressively at every layer (Redis for DB reads, CDN for static assets). Add read replicas to split DB read/write load. Partition your database (sharding) when a single DB can't handle volume. Use async message queues to decouple heavy processing from user-facing requests. Use a CDN to serve content from edge nodes near users. Design stateless services so any instance can handle any request — no session affinity needed. And add auto-scaling (Kubernetes HPA, AWS Auto Scaling) so capacity adjusts automatically under load.

Deep-Dive Topics

Redis Caching Strategies

Cache-Aside (Lazy Loading):

The application owns the cache logic. On read: check cache → miss → read DB → write to cache → return. Most common pattern. Cache only contains data actually requested. Downside: first request always misses (cold start), and if cache crashes, all traffic hits DB until it warms up again.

Read-Through:
Cache sits in front of DB. Application only talks to cache. On miss, the cache itself fetches from DB and populates. Transparent to the app. Good for read-heavy systems. Used in providers like AWS ElastiCache with DAX (DynamoDB).

Write-Through:
Every write goes to cache AND DB synchronously. Cache is always consistent with DB. Downside: write latency increases, and you cache data that may never be read again (write amplification). Pairs well with read-through.

Write-Behind (Write-Back):
Write goes to cache immediately; DB is updated asynchronously. Fast writes, low latency. Risk: if Redis crashes before the async flush, you lose data. Use only when you can tolerate occasional data loss (analytics counters, non-critical metrics).

Refresh-Ahead:
Cache proactively refreshes data before TTL expires, based on predicted access patterns. Reduces cache miss latency spikes. Complex to implement correctly — requires knowing what to pre-fetch.

TTL Strategy:
Every cached key should have a TTL. Short TTL = more DB hits but fresher data. Long TTL = fewer DB hits but stale data risk. For critical data (claim status), use short TTL + explicit invalidation on update. For reference data (payer list), longer TTL is fine.

Cache Invalidation:
The hardest problem in caching. Three approaches: TTL expiry (simplest, acceptable staleness), event-driven invalidation (write to DB → publish event → cache listener deletes the key — strong consistency), and versioned keys (append a version to the key, bump on write — old keys naturally expire).

Redis Eviction Policies (when memory is full):

- `allkeys-lru` — evict least recently used key from all keys. Good general default.
- `volatile-lru` — LRU only among keys with TTL set.
- `allkeys-lfu` — evict least frequently used. Better for skewed access patterns.
- `volatile-ttl` — evict key with shortest remaining TTL.
- `noeviction` — reject writes. Safe for session stores where silent loss is unacceptable.

Consistency Models

Consistency in distributed systems describes how up-to-date and synchronized data is across nodes after a write.

Strong Consistency:
After a write completes, every subsequent read from any node sees that write. System behaves as if there's one copy of data. Simple to reason about. Expensive — requires coordination (locks, consensus protocols like Raft/Paxos). Used in: relational databases with synchronous replication, Google Spanner, ZooKeeper. Use when correctness is non-negotiable (financial transactions, inventory).

Eventual Consistency:
After a write, replicas will eventually converge to the same value — but reads during the propagation window may return stale data. No coordination needed — writes are fast. Used in: Cassandra, DynamoDB (default), DNS. Use when availability and performance matter more than instantaneous accuracy (social media likes, shopping cart recommendations, activity feeds).

Read-Your-Writes Consistency:
A specific user always sees their own writes immediately, even if other users might see stale data. Common in user-facing apps — if you update your profile, you see the change instantly. Implemented by routing a user's reads to the replica they last wrote to, or using sticky sessions.

Monotonic Read Consistency:
Once a client reads a value, it never reads an older value for the same key. Prevents the bizarre scenario where you read version 5, then read version 3. Achieved by pinning a client to a specific replica.

Causal Consistency:
Operations that are causally related appear in order to all nodes. If you post a comment in response to a post, anyone who sees the comment also sees the original post. Weaker than strong, stronger than eventual. Used in collaborative tools, distributed databases like MongoDB (causally consistent sessions).

CAP Theorem context:
In a network partition (P), you must choose between Consistency (C) and Availability (A). SQL DBs with synchronous replication choose CP (reject writes if can't confirm all replicas). Cassandra/DynamoDB choose AP (stay available, accept temporary inconsistency). In

practice, network partitions are rare — the real trade-off is latency vs consistency (PACELC theorem).

Scalable System Design Basics

1. Stateless Services:
Keep no session or user state inside the service process. Store state in Redis or a DB. Any instance can serve any request — this is what makes horizontal scaling possible. If your service is stateful (holds in-memory session), you're stuck with sticky sessions and limited scaling.

2. Horizontal Scaling:
Add more instances rather than a bigger machine. Requires a load balancer in front. Works because services are stateless. Combine with auto-scaling (Kubernetes HPA scales pods based on CPU/memory or custom metrics like queue depth).

3. Database Scaling:

- **Read replicas** — offload reads to replicas; primary handles writes only
- **Connection pooling** — services share a pool of DB connections (PgBouncer, HikariCP) to avoid connection exhaustion
- **Sharding** — partition data across multiple DB instances by a shard key (userId, claimId). Each DB owns a subset. Complex but necessary at extreme scale.
- **Caching** — reduce DB load by caching hot reads in Redis

4. Asynchronous Processing:
Move non-user-facing work off the request path. Claim submitted → return 202 Accepted → process eligibility check async via Kafka. Keeps API latency low and decouples heavy processing.

5. CDN for Static Assets:
Never serve images, JS, CSS from your origin servers. Push to CloudFront / Akamai. Edge nodes serve users from the closest geographic location.

6. Database Indexing:
The most impactful single optimization in most systems. Every column used in WHERE, JOIN, ORDER BY should be indexed. Without indexes, queries do full table scans — O(n). With indexes — O(log n). Over-indexing hurts write performance. Profile your slow query log and add indexes strategically.

7. Pagination:
Never return unbounded result sets. Cursor-based pagination (return a `nextCursor` token) scales better than offset pagination (OFFSET 10000 still scans 10000 rows).

API Design Best Practices

1. Use REST correctly:

- Nouns for resources, not verbs: `/claims` not `/getClaims`
- HTTP methods carry the verb: `GET /claims`, `POST /claims`, `PUT /claims/{id}`, `DELETE /claims/{id}`
- `GET` is idempotent and safe. `PUT`/`DELETE` are idempotent. `POST` is not.
- Use `PATCH` for partial updates, `PUT` for full replacement.

2. Versioning:
Always version your APIs from day one. `/api/v1/claims`. When you need breaking changes, release `/v2` — old clients keep working. Never change a live API's contract without versioning.

3. Status codes — use them correctly:

- `200 OK`, `201 Created`, `202 Accepted` (async processing started)
- `400 Bad Request` (validation failure), `401 Unauthorized` (no/bad token), `403 Forbidden` (token valid, no permission), `404 Not Found`, `409 Conflict`
- `500 Internal Server Error`, `503 Service Unavailable`
- Never return `200` with an error body — that breaks every monitoring tool.

4. Consistent error responses:
Always return a structured error body — not just a status code.

```
{
  "error": "CLAIM_NOT_FOUND",
  "message": "No claim found with id claim-123",
  "timestamp": "2026-06-05T10:00:00Z",
```

```
  "traceId": "abc-xyz-123"
}
```

5. Idempotency keys:

For POST endpoints that create or mutate (especially payments/claims), accept an `Idempotency-Key` header. If the client retries due to network timeout, the server recognizes the key and returns the original response instead of creating a duplicate.

6. Pagination, filtering, sorting:

Expose these consistently: `GET /claims?status=SUBMITTED&page=2&size=20&sort=createdAt,desc`. Never return unbounded lists.

7. Rate limiting:

Return `429 Too Many Requests` with a `Retry-After` header. Expose limits in response headers (`X-RateLimit-Limit`, `X-RateLimit-Remaining`).

8. Security basics:

Always validate and sanitize inputs. Use HTTPS only. Authenticate every request (JWT/OAuth2). Authorize by role/scope, not just authentication. Never expose internal error stack traces in API responses.

Distributed Systems Basics

1. Fallacies of Distributed Computing:

The network is NOT reliable. Latency is NOT zero. Bandwidth is NOT infinite. The network is NOT secure. Topology changes. There is NOT one administrator. Transport cost is NOT zero. The network is NOT homogeneous. Every distributed system design must account for these.

2. CAP Theorem:

A distributed system can guarantee at most 2 of 3: Consistency (every read sees the latest write), Availability (every request gets a response), Partition Tolerance (system continues if nodes can't communicate). Since network partitions happen, real systems choose CP or AP. CP: strong consistency, reject requests during partition (ZooKeeper, HBase). AP: stay available, accept eventual consistency (Cassandra, DynamoDB).

3. Consensus:

Getting multiple nodes to agree on a value in the presence of failures. Solved by Raft (used in etcd, CockroachDB) and Paxos. Used for leader election, distributed locks, and replicated

logs. Important to know why you can't just use a DB for distributed locking — you need fencing tokens and lease-based approaches.

4. Idempotency:
An operation that produces the same result whether executed once or many times. Critical in distributed systems where retries happen. Design all mutations to be idempotent — use idempotency keys, natural uniqueness constraints, or event deduplication via Redis `SET NX`.

5. Circuit Breaker:
If Service B is failing, stop sending requests to it — fail fast instead of queueing up. After a timeout, send a test request. If B recovers, close the circuit. Implemented in Spring Boot via Resilience4j. Prevents cascading failure where one slow service takes down the entire call chain.

6. Saga Pattern:
Managing distributed transactions across microservices without a global 2-phase commit. A saga is a sequence of local transactions, each publishing an event. If step 3 fails, compensating transactions undo steps 1 and 2. Two flavors: Choreography (services react to events) and Orchestration (a central saga orchestrator commands each step). Your Claims → Eligibility → Adjudication flow is a candidate for this.

7. Observability (the 3 pillars):

- **Logs** — structured JSON logs with correlation IDs, searchable in ELK/CloudWatch
- **Metrics** — counters, gauges, histograms (Prometheus + Grafana, CloudWatch Metrics)
- **Traces** — distributed traces that follow a request across services (AWS X-Ray, Jaeger, Zipkin)
  Without observability, debugging distributed systems is nearly impossible.

8. Health checks and readiness probes:
Every service exposes `/health` (liveness — is the process alive?) and `/ready` (readiness — is it ready to accept traffic?). Kubernetes uses these to restart crashed pods and hold traffic during startup. Spring Boot Actuator gives you these for free.

6/5/2026, 10:23:27 AM

webhooks basics
Grpc Basics
exception handling patterns
performance tuning basics

What happens if traffic increases 10x?
Where will bottleneck occur?
How will data be stored?
What fails first?
How do we scale?

how to Practice thinking like:
“How does this feature scale?”
“What happens if traffic increases 10x?”
“How to debug production issue?”
“How to design fault-tolerant system?”

6/5/2026, 5:19:13 PM

System Design — Round 2 Notes

1. Webhooks Basics

What it is: A webhook is a way for one system to notify another system when something happens, using an HTTP POST request. Instead of the receiver polling ("did anything change?"), the sender pushes a notification the moment the event occurs. It's sometimes called a "reverse API" — the server calls you, not the other way around.

Why we use it: Polling is wasteful. If you poll every 5 seconds to check if a claim status changed, you're making thousands of empty requests. A webhook fires once, exactly when the event happens. Lower latency, lower cost, simpler consumer logic.

Simple example: A payer system processes your claim and its status changes to `ADJUDICATED`. It sends an HTTP POST to `https://availity.com/webhooks/claim-status` with the payload `{ "claimId": "C123", "status": "ADJUDICATED" }`. Your service

receives it, updates Aurora, publishes a Kafka event, and notifies the provider. No polling needed.

How to implement webhooks correctly:

- **Receiver must respond fast.** Return `200 OK` immediately. Do not process inline — push to a queue and process async. If you take 10 seconds to respond, the sender will think you failed and retry.
- **Verify the sender.** Senders sign the payload with an HMAC-SHA256 signature using a shared secret. You recompute the signature on your side and compare. Reject if it doesn't match — otherwise anyone can POST fake events to your endpoint.
- **Handle retries and duplicates.** Senders retry on non-200 responses. Your receiver must be idempotent — use the event ID to deduplicate (store processed event IDs in Redis with TTL).
- **Return correct status codes.** `200` = received. `400` = bad payload (don't retry). `500` = server error (do retry). Never return `200` for an invalid payload — you'll silently lose events.
- **Secure the endpoint.** HTTPS only. Validate the `Content-Type`. Rate limit the endpoint. Whitelist sender IPs if the sender publishes them.

Webhook vs Polling vs WebSocket:

	Webhook	Polling	WebSocket
Direction	Server pushes to client	Client pulls from server	Bidirectional, persistent
Latency	Near real-time	Delayed by poll interval	Real-time
Use case	Event notifications between services	Batch status checks	Live dashboards, chat
Complexity	Medium (needs public endpoint, idempotency)	Simple	High (stateful connection)

2. gRPC Basics

What it is: gRPC is a high-performance RPC (Remote Procedure Call) framework built by Google. Instead of REST (text-based JSON over HTTP/1.1), gRPC uses Protocol Buffers (binary serialization) over HTTP/2. You define your service and data types in a `.proto` file, and gRPC auto-generates client and server code in any language.

Why we use it: JSON is human-readable but large and slow to parse. Protobuf is binary — 3–10x smaller and much faster to serialize/deserialize. HTTP/2 gives you multiplexing (multiple requests over one connection), header compression, and streaming. gRPC is ideal for internal service-to-service communication where you control both ends.

Simple example `.proto` file:

```
syntax = "proto3";

service ClaimsService {
  rpc SubmitClaim (ClaimRequest) returns (ClaimResponse);
  rpc StreamClaimUpdates (ClaimRequest) returns (stream ClaimEvent);
}

message ClaimRequest {
  string claim_id = 1;
  string patient_id = 2;
  double amount = 3;
}

message ClaimResponse {
  string status = 1;
  string tracking_id = 2;
}
```

gRPC generates Java stubs from this. Your service implements the interface. The client calls it like a local method — no HTTP path, no JSON parsing, no manual deserialization.

Four communication patterns in gRPC:

- **Unary:** One request, one response. Like REST. `submitClaim(request) → response`
- **Server streaming:** One request, stream of responses. Server pushes multiple messages back. `streamClaimUpdates(request) → stream of ClaimEvent`

- **Client streaming:** Client sends a stream of requests, server returns one response. Useful for bulk uploads.
- **Bidirectional streaming:** Both sides stream simultaneously. Real-time two-way communication. Used in live collaboration, live monitoring.

REST vs gRPC:

Aspect	REST	gRPC
Protocol	HTTP/1.1	HTTP/2
Data format	JSON (text)	Protobuf (binary)
Schema	Optional (OpenAPI)	Mandatory (.proto)
Code generation	Manual or via tools	Auto-generated stubs
Browser support	Native	Needs grpc-web proxy
Streaming	Limited (SSE, WebSocket)	First-class support
Use case	Public APIs, browser clients	Internal microservices

When to use gRPC: Internal microservice communication, polyglot services (Java calling Go calling Python), high-throughput data pipelines, streaming use cases. Not ideal for public APIs where clients are browsers — REST is still the standard there.

Spring Boot gRPC: Use `grpc-spring-boot-starter`. Define `.proto`, annotate your service with `@GrpcService`, inject clients with `@GrpcClient`. Works alongside your existing Spring context.

---

3. Exception Handling Patterns

Why this matters in interviews: Interviewers want to see that you think beyond the happy path. Good exception handling is the difference between a system that recovers gracefully and one that loses data silently or cascades failures.

Layer 1 — Global Exception Handler (Spring Boot):

Use `@RestControllerAdvice` to centralize all exception handling in one place. No try-catch in every controller.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ClaimNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleNotFound(ClaimNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("CLAIM_NOT_FOUND",
ex.getMessage()));
    }

    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ErrorResponse>
handleValidation(ValidationException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse("VALIDATION_FAILED",
ex.getMessage()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex)
{
        log.error("Unhandled exception", ex);  // log stack trace
here, not in response
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("INTERNAL_ERROR", "Something
went wrong"));
    }
}
```

Never expose stack traces in API responses. Log them internally. Return structured error bodies always.

Layer 2 — Custom Exception Hierarchy:

```
RuntimeException
├── BusinessException (your base)
│   ├── ClaimNotFoundException      → 404
│   ├── DuplicateClaimException     → 409
│   ├── PayerNotEligibleException   → 422
│   └── ClaimValidationException    → 400
└── SystemException
    ├── ExternalServiceException    → 502
    └── DatabaseException           → 500
```

Typed exceptions let your global handler map cleanly to HTTP status codes without if-else chains. It also makes code readable — `throw new ClaimNotFoundException(id)` is self-documenting.

Layer 3 — Retry Pattern:

Not all failures are permanent. Network blips, DB connection pool exhaustion, and external API timeouts are transient — retry helps. Spring Retry or Resilience4j handles this cleanly.

```
@Retryable(
    value = { ExternalServiceException.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 500, multiplier = 2)  // 500ms,
1000ms, 2000ms
)
public EligibilityResponse checkEligibility(String memberId) {
    return eligibilityClient.check(memberId);
}

@Recover
public EligibilityResponse fallback(ExternalServiceException ex,
String memberId) {
    log.error("Eligibility check failed after retries for member
{}", memberId, ex);
    return EligibilityResponse.unknown();  // graceful degradation
}
```

Exponential backoff with jitter prevents thundering herd — if 100 threads all retry at the same moment, you amplify the problem.

Layer 4 — Circuit Breaker (Resilience4j):

```
@CircuitBreaker(name = "eligibilityService", fallbackMethod =
"eligibilityFallback")
public EligibilityResponse checkEligibility(String memberId) {
    return eligibilityClient.check(memberId);
}
```

States: CLOSED (normal) → OPEN (failing, reject immediately) → HALF-OPEN (test one request). Prevents cascading failure. If eligibility service is down, claims service fails fast instead of hanging on every request.

Layer 5 — Kafka Consumer Exception Handling:

```
@KafkaListener(topics = "claim-submitted")
public void handleClaim(ClaimEvent event) {
    try {
        claimProcessor.process(event);
    } catch (RetryableException e) {
        throw e;  // Kafka will retry
    } catch (NonRetryableException e) {
        log.error("Non-retryable error, sending to DLT", e);
        deadLetterTemplate.send("claim-submitted.DLT", event);
    }
}
```

Dead Letter Topic (DLT) is your safety net. Failed messages park there for investigation and manual replay. Never let a consumer silently swallow exceptions — you'll lose messages.

Layer 6 — What NOT to do:

- Never catch `Exception` broadly and swallow it silently with just a log line
- Never return `200 OK` with an error body
- Never expose internal error details (DB connection strings, stack traces) to clients
- Never retry non-idempotent operations without idempotency keys

- Never let a circuit breaker fail open without a fallback

---

4. Performance Tuning Basics

Where to look first: Measure before you optimize. Use profilers, APM tools (Datadog, AWS X-Ray, New Relic), slow query logs, and JVM metrics. Optimizing without measuring is guessing.

Database — the most common bottleneck:

- Add indexes on columns in WHERE, JOIN, ORDER BY. Check `EXPLAIN` / `EXPLAIN ANALYZE` to see if queries use indexes.
- Use connection pooling (HikariCP — already in Spring Boot). Without it, every request opens a new DB connection — DB crashes under load.
- Avoid N+1 queries. Fetching 100 claims then doing 100 separate queries for each claim's payer is N+1. Use JOIN or batch fetch.
- Use read replicas for reporting queries. Never run heavy analytical queries on your primary write DB.
- Paginate all list queries. `SELECT * FROM claims` with no LIMIT on a large table will kill your DB.

JVM / Spring Boot:

- Use async processing (`@Async`, reactive) for I/O-bound operations. Don't block threads waiting for DB or HTTP calls.
- Tune thread pool sizes — too small: requests queue up. Too large: context switching overhead. Profile under realistic load.
- Watch heap size and GC behavior. Frequent full GCs cause stop-the-world pauses and latency spikes. Use G1GC or ZGC for large heaps.
- Avoid creating large objects in hot paths. Object allocation pressure forces more frequent GC.

Caching:

- Cache at the right layer — Redis for DB queries, local Caffeine cache for config/reference data that changes rarely.

- Cache hit rate matters. If hit rate is below 80%, your cache key or TTL strategy is wrong.
- Don't cache mutable, high-cardinality data — you'll waste memory and cause stale data bugs.

HTTP / API layer:

- Enable HTTP keep-alive and connection pooling in your REST clients (RestTemplate/WebClient). Opening a new TCP connection per request is expensive.
- Compress responses with gzip — especially for large JSON payloads. Spring Boot enables this with one config line.
- Use async endpoints for long-running operations. Return `202 Accepted` with a polling URL instead of blocking the HTTP thread.

Kafka:

- Batch producer sends (`linger.ms`, `batch.size`) — small batches mean many network round trips.
- Increase consumer parallelism by adding partitions and consumer instances. One consumer per partition max.
- Monitor consumer lag (offset lag) — if lag grows, consumers can't keep up with producers. Scale out consumers.

Measuring:

- Response time percentiles: P50, P95, P99. Average hides the tail latency that users actually experience.
- Throughput: requests per second at which latency degrades.
- Error rate: any spike in 5xx under load reveals a bottleneck.
- Saturation: CPU %, memory %, DB connections used, thread pool queue depth.

Production Scenario Thinking

What Happens if Traffic Increases 10x?

Walk through each layer of the stack systematically. This is exactly the mental model interviewers want to see.

Layer 1 — Load Balancer:
ALB/Nginx scales automatically in cloud setups. Rarely the bottleneck unless you have a misconfigured single instance. Check: connection limits, SSL termination overhead.

Layer 2 — Application Servers:
Stateless Spring Boot pods behind a load balancer → scale horizontally. Kubernetes HPA auto-adds pods when CPU crosses threshold. What breaks: thread pool exhaustion (incoming requests queue up if all threads are waiting on DB/IO), connection pool exhaustion (if each pod opens 10 DB connections and you go from 5 pods to 50, you need 500 DB connections — does your DB support that?).

Layer 3 — Cache:
Redis is fast and handles high concurrency well. What breaks: cache stampede — if TTL expires and 1000 requests simultaneously miss cache and all hit DB at once. Fix with mutex locking or probabilistic refresh. Also watch: Redis memory limit — if cache is full and eviction policy is wrong, you evict hot keys.

Layer 4 — Database — this fails first, almost always:
DB is stateful and hardest to scale. At 10x traffic: connection pool saturation (add PgBouncer/ProxySQL as a connection pooler between app and DB), slow query volume increases, replication lag builds up on read replicas, lock contention increases on hot rows. Fix: read replicas for read traffic, query optimization, caching hot reads, eventually sharding.

Layer 5 — Message Queue (Kafka):
Producers rarely back up — Kafka absorbs spikes into the log. What breaks: consumer lag — if consumers can't process fast enough, lag grows and processing delay increases. Fix: scale consumers horizontally (add instances up to partition count), increase partition count if you need more parallelism than current partitions allow.

Layer 6 — External Services:
At 10x traffic your call volume to external APIs (payer eligibility endpoints) also increases 10x. External rate limits get hit. Fix: cache external responses where possible, implement circuit breakers, batch calls if the API supports it.

Where the bottleneck occurs — in order of likelihood:

1. Database (connection pool, slow queries, lock contention) — always first

2. External service rate limits
3. Application thread pool exhaustion
4. Kafka consumer lag
5. Cache stampede on key expiry
6. Load balancer (rare in cloud setups)

How Will Data Be Stored at Scale?

Transactional data (claims, users, billing):
Start with a single Aurora MySQL primary + read replicas. As write volume grows: vertical scaling first (larger RDS instance), then read replicas for read traffic, then consider Aurora Serverless for variable workloads. At extreme scale: sharding by `claimId` hash or `tenantId`. Each shard is an independent DB — routing logic at application layer.

Event/audit data:
Never store high-volume event logs in your transactional DB. Write to Kafka → consume → store in S3 (cheap, durable) or Elasticsearch (searchable). Use columnar formats (Parquet) for analytical queries on S3 via Athena.

Document data (variable schema):
MongoDB scales horizontally via sharding. Shard key choice matters — poor shard key (low cardinality, like `status`) creates hot shards. Good shard key distributes writes evenly (`claimId` or compound key).

Session / cache data:
Redis Cluster for horizontal scaling. Data partitioned automatically across nodes. Add Redis Sentinels or use Redis Cluster mode for high availability.

Cold data:
Data older than 90 days rarely accessed — archive to S3 Glacier or move to a separate analytics DB. Don't let your transactional DB grow unbounded.

What Fails First?

In almost every system under load stress:

1. **Database connections** — pool exhausts, new requests hang waiting for a connection

2. **Slow queries** — under 10x read load, unindexed queries that were "fast enough" at 1x become table scans on a hot DB
3. **External API rate limits** — third-party payer APIs don't scale with you
4. **Application thread pool** — if threads are blocked on slow DB/IO, incoming HTTP requests queue up and eventually timeout
5. **Memory pressure / GC pauses** — 10x load means 10x object allocation rate; GC pauses increase, latency spikes appear in P99
6. **Kafka consumer lag** — consumers fall behind producers if downstream processing (DB writes, external calls) is slow

The pattern is always: **stateful systems fail before stateless ones.** Your Spring Boot service is easy to scale. Your DB is not.

How Do We Scale?

Immediate (horizontal scaling):
Add more application instances. Kubernetes HPA does this automatically. Works instantly for stateless services.

Short-term (caching and query optimization):
Add Redis caching for hot DB reads. Add indexes on slow queries. Add DB read replicas and route read traffic there. These steps typically handle 5–10x without architectural changes.

Medium-term (async and decoupling):
Move synchronous flows to async (Kafka). Return 202 immediately, process in background. This absorbs traffic spikes into queue buffers and gives you time to process at your own rate.

Long-term (data tier scaling):
Connection pooler (PgBouncer) in front of DB. DB sharding for write-heavy workloads. CQRS — separate write model (normalized, transactional) from read model (denormalized, fast). Archive old data to S3. Use Elasticsearch for search instead of `LIKE` queries on MySQL.

Architectural (if you outgrow everything above):
Multi-region active-passive or active-active. Global load balancing. Data partitioned by geography. Cell-based architecture (each cell handles a subset of tenants and fails independently).

How to Think Like a Senior Engineer

Mental Model 1: "How Does This Feature Scale?"

Every time you build or review a feature, run it through this checklist before you consider it done:

Ask about the data:

- How many records exist today? In 1 year at 10x growth?
- Is there a query that touches all of them (no WHERE clause, no index)?
- Is there a table that gets written to on every user action?

Ask about the read/write ratio:

- Is this read-heavy (serve from cache/read replica) or write-heavy (need partitioning)?
- Can reads tolerate slight staleness, or does the user need the freshest data?

Ask about concurrency:

- What happens if 100 users do this simultaneously?
- Is there a shared lock, a shared counter, a shared row they all update?
- Race condition if two requests process the same claimId at the same time?

Ask about failure:

- If the downstream service is down, what happens to this request?
- Is there a retry? Is it idempotent?
- Does a failed request leave data in a partial state?

Practical habit: Before every PR, write one line in the PR description: "Scaling concern: X". Even if the answer is "none identified", the act of asking trains the instinct.

Mental Model 2: "What Happens if Traffic Increases 10x?"

Walk the call stack from browser to DB and ask at each layer: "Can this handle 10x?"

```
Browser/Client
    ↓ × 10
CDN / API Gateway          → Can it route 10x? Usually yes (managed
service)
    ↓ × 10
Load Balancer              → Can it distribute 10x? Usually yes
    ↓ × 10
Application Server (pods)  → Can it handle 10x? Add pods (HPA)
    ↓ × 10
Cache (Redis)              → Can it serve 10x reads? Usually yes,
watch stampede
    ↓ (only cache misses, maybe 2x of original)
Database                   → CAN IT? This is where you slow down and
think.
    ↓ × 10
External APIs              → Rate limits? Timeout budgets?
```

The DB question breaks into:

- Connections: 10x pods × connections per pod = does max_connections allow this?
- Query time: does EXPLAIN show a full table scan that's acceptable at 1x but kills you at 10x?
- Lock contention: is there a hot row (a status counter, a sequence, a shared config row) that serializes writes?
- Replication lag: at 10x writes, do read replicas fall behind and serve stale data?

Answering these for any system — even hypothetically — is exactly what system design interviewers want to hear.

Mental Model 3: "How to Debug a Production Issue?"

Follow this structured approach every time. Don't guess. Narrow.

Step 1 — Define the symptom precisely.
"Something is slow" is not actionable. "P99 latency on `POST /claims` spiked from 200ms to 4s at 14:32 IST, error rate is 0%, CPU is normal" is.

Step 2 — Establish the timeline.
When did it start? Was there a deployment, a config change, a traffic spike, or a cron job that ran? Correlation with changes is the fastest path to root cause.

Step 3 — Check the 4 golden signals:

- Latency (is it slow?)
- Traffic (is volume normal, higher, lower?)
- Errors (what's the error rate and which errors?)
- Saturation (CPU, memory, DB connections, thread pool — what's at capacity?)

Step 4 — Isolate the layer.
Is it slow at the API Gateway? Then it's not the app. Slow at the app but DB queries are fast? It's the app logic or thread pool. DB queries slow? Check EXPLAIN and slow query log. External service calls slow? Check circuit breaker metrics.

Step 5 — Follow the trace.
Distributed tracing (X-Ray, Jaeger) shows you which span in the request took the time. A 4-second request might be: 10ms Spring logic + 3900ms DB query + 90ms response serialization. The trace tells you exactly where.

Step 6 — Form a hypothesis, verify, fix.
"I think this query on `claims` table has no index on `submitted_date` and is doing a full scan under high load." → Check `EXPLAIN`. → Confirm. → Add index. → Deploy. → Confirm latency drops.

Step 7 — Write a postmortem.
Timeline, root cause, impact, fix, and — most importantly — what systemic change prevents this class of issue from recurring. This is what separates senior engineers from mid-level engineers.

Mental Model 4: "How to Design a Fault-Tolerant System?"

Fault tolerance means the system continues to function (possibly in a degraded mode) when components fail. The core principle: **assume everything fails, design for it explicitly.**

Eliminate single points of failure:
Every component that exists as a single instance is a SPOF. Multiple app instances behind a

load balancer. Multi-AZ DB (Aurora automatically replicates across AZs). Redis with Sentinel or Cluster mode. Kafka with replication factor ≥ 3. If any component going down takes your system with it, that's a SPOF to fix.

Graceful degradation:
When a non-critical dependency fails, return a degraded response rather than an error. Eligibility service is down → accept the claim, mark it pending eligibility check, process later. Notification service is down → claim still submits successfully, notification retried via Kafka DLT. Users should never see a hard failure for a background service's downtime.

Timeouts everywhere:
Every external call — DB query, HTTP call, Redis call — needs an explicit timeout. Without timeouts, a slow dependency holds your thread forever, thread pool exhausts, and your service appears down even though it's alive. Set connect timeout, read timeout, and circuit breaker threshold for every integration.

Idempotent operations:
Design mutations to be safely retried. Use idempotency keys on claim submissions. Make Kafka consumers idempotent — receiving the same event twice should not create duplicate data. This lets you retry aggressively without fear of data corruption.

Bulkhead pattern:
Separate thread pools for separate concerns. Your thread pool for external payer API calls should not share threads with your claim submission flow. If payer API is slow and saturates its thread pool, claim submissions are unaffected. This is the bulkhead — like watertight compartments in a ship.

Test your failure modes:
Chaos engineering — deliberately kill instances, inject latency, drop network packets — and verify the system behaves correctly. If you only find out your circuit breaker is misconfigured during a real outage, that's too late. Netflix's Chaos Monkey approach, even informally, reveals assumptions you didn't know you were making.

The three questions for every design decision:

1. What happens when this component is down?
2. What happens when this component is slow?
3. What happens when this component returns corrupt/unexpected data?

If you can answer all three with "the system degrades gracefully and recovers", the design is fault-tolerant.

6/5/2026, 5:21:26 PM

How to do db partition what are the ways

6/6/2026, 6:50:40 AM

Database Partitioning — Complete Guide

What is Partitioning?

Partitioning means splitting a large table into smaller, manageable pieces. The data is divided across multiple physical storage units, but the application queries it as if it's one table. The DB engine (or your application) figures out which partition to read from or write to.

Why partition:

- A table with 500 million rows is slow to query even with indexes — the index itself becomes huge
- Writes on a single table create lock contention
- Archiving old data becomes hard when everything is in one table
- Backup and maintenance operations lock the whole table

Two Broad Categories

```
Partitioning
├── Vertical Partitioning     → split by columns
└── Horizontal Partitioning   → split by rows (this is what people mean 99% of the time)
    ├── Range Partitioning
    ├── List Partitioning
    ├── Hash Partitioning
    ├── Key Partitioning
    └── Composite Partitioning
```

Vertical Partitioning

Split a wide table into narrower tables by column group. All rows stay together, but columns are separated.

Example: Your `claims` table has 40 columns — 8 are queried on every read (hot columns), 32 are rarely accessed audit/detail columns.

```
claims table (40 cols, queried millions/day)
        ↓ vertical partition
claims_core   (claimId, patientId, status, amount, payerId,
createdAt, updatedAt, tenantId)
claims_detail (claimId, diagnosisCodes, procedureCodes,
renderingProvider, billingNotes, ...)
```

Why: `claims_core` fits more rows per DB page → fewer I/O operations → faster reads. `claims_detail` is fetched only when needed (claim detail screen, not list screen).

This is essentially normalization. You already do a form of this in your CQRS setup — write model has full detail, read model has projected summary.

Horizontal Partitioning (Sharding the table)

Split by rows. Each partition holds a subset of rows based on a partition key. Same columns everywhere, different data.

---

1. Range Partitioning

Divide rows based on a range of values in a column — most commonly a date column.

```sql
-- MySQL example
CREATE TABLE claims (
    claimId     VARCHAR(36),
    submittedAt DATE,
    status      VARCHAR(20),
    amount      DECIMAL(10,2)
)
PARTITION BY RANGE (YEAR(submittedAt)) (
    PARTITION p2022 VALUES LESS THAN (2023),
```

```
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

Query behavior: `SELECT * FROM claims WHERE submittedAt BETWEEN '2024-01-01' AND '2024-12-31'` — DB only scans partition `p2024`. This is called **partition pruning**.

Best for: Time-series data, audit logs, billing cycles, anything with date-based queries and archival needs.

Pros:

- Natural fit for date-range queries
- Easy to archive — `ALTER TABLE claims DROP PARTITION p2022` deletes a whole year instantly, no DELETE statement scanning millions of rows
- Partition pruning makes range queries very fast

Cons:

- Hot partition problem — all current writes go to the latest partition. 2025 partition gets hammered, older ones are idle.
- If your queries are not date-range based, no benefit

---

2. List Partitioning

Divide rows based on a specific set of discrete values — like region, status, or tenant.

```
CREATE TABLE claims (
    claimId  VARCHAR(36),
    region   VARCHAR(20),
    status   VARCHAR(20),
    amount   DECIMAL(10,2)
)
PARTITION BY LIST COLUMNS (region) (
    PARTITION p_east   VALUES IN ('US-EAST-1', 'US-EAST-2'),
```

```
    PARTITION p_west   VALUES IN ('US-WEST-1', 'US-WEST-2'),
    PARTITION p_central VALUES IN ('US-CENTRAL-1'),
    PARTITION p_india  VALUES IN ('AP-SOUTH-1')
);
```

Best for: Multi-tenant systems partitioned by tenantId, regional data isolation, partitioning by a known fixed set of categories.

Pros:

- Clean data isolation per category
- Regulatory compliance — Indian tenant data physically stays in p_india partition

Cons:

- Adding new values requires ALTER TABLE to add a partition
- Uneven data distribution if some regions have far more data

---

3. Hash Partitioning

Apply a hash function to the partition key. Row goes to partition `hash(key) % N`.

```
CREATE TABLE claims (
    claimId  BIGINT,
    status   VARCHAR(20),
    amount   DECIMAL(10,2)
)
PARTITION BY HASH (claimId)
PARTITIONS 8;
-- DB automatically routes claimId=1001 to partition (1001 % 8) =
partition 1
```

Best for: When you have no natural range or list grouping but need to spread load evenly. High-write tables where you want to distribute write contention across partitions.

Pros:

- Very even data distribution — no hot partitions

- No need to define partition ranges upfront

Cons:

- Partition pruning only works if you query by exact claimId (the hash key). Range queries (`WHERE claimId BETWEEN 100 AND 200`) scan all partitions — no benefit.
- Hard to archive — data from all time periods is mixed across all partitions
- If you change the number of partitions (8 → 16), you have to rehash everything — massive data migration

---

4. Key Partitioning

Similar to hash partitioning but the DB engine provides the hash function (usually MySQL's internal hash). You just specify the column.

```
PARTITION BY KEY (claimId)
PARTITIONS 8;
```

Works well with primary keys. Less control than hash but simpler to set up.

---

5. Composite (Sub) Partitioning

Combine two strategies. Partition by range first, then hash within each range partition.

```
PARTITION BY RANGE (YEAR(submittedAt))
SUBPARTITION BY HASH (claimId)
SUBPARTITIONS 4 (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026)
);
```

Each year partition is further split into 4 hash sub-partitions. You get both date-range pruning AND even write distribution within the year.

Best for: Very large tables where a single strategy isn't enough. Common in data warehouses and analytics systems.

Sharding — Application-Level Horizontal Partitioning

When your table grows beyond what one DB server can hold, sharding distributes partitions across **multiple physical database servers**. This is the big leap — from partitioning within one DB to partitioning across many DBs.

```
Single DB (one server)
  ├── claims_p2023
  ├── claims_p2024
  └── claims_p2025

Sharded across servers:
  DB Server 1  → Shard A (tenantId 0000–3333)
  DB Server 2  → Shard B (tenantId 3334–6666)
  DB Server 3  → Shard C (tenantId 6667–9999)
```

Shard key selection — the most critical decision:

A bad shard key ruins everything. Criteria for a good shard key:

- **High cardinality** — enough distinct values to distribute data evenly
- **Even write distribution** — no hot shard (one shard getting all writes)
- **Query locality** — most queries should hit one shard, not all of them
- **Immutable** — once a row is sharded, changing the shard key means migrating the row

Common shard keys:

Shard Key	Good for	Risk
`tenantId`	Multi-tenant SaaS	Large tenants create hot shards
`userId`	Social platforms	Celebrity users (high-follower accounts) skew shards
`claimId` hash	Even distribution	Cross-shard queries become scatter-gather
`geographyId`	Regional compliance	Uneven regional traffic
`createdAt`	Time-series	Recent shard always hot

Consistent hashing: Used to avoid rehashing everything when you add a shard. Instead of `hash(key) % N`, keys are mapped to a ring and each server owns a range of the ring. Adding a server only moves a fraction of keys, not all of them. Used in Cassandra, DynamoDB, Redis Cluster.

Partitioning Strategies Comparison

Strategy	Split by	Best query pattern	Hot partition risk	Archive friendly
Range	Value ranges (date)	Date range queries	Yes — latest range	Yes — drop old partition
List	Discrete values (region)	Exact category filter	Maybe — uneven categories	Partial
Hash	Hash of key	Point lookups by key	No — even distribution	No — data mixed
Composite	Range + Hash	Date range + point lookup	Low	Yes
Sharding	Across DB servers	Depends on shard key	Depends on shard key	Depends

---

Partitioning in Your Stack (Practical Relevance)

Aurora MySQL: Supports all MySQL partition types natively. For your Claims table, range partitioning by `submittedAt` is the most natural fit — queries are almost always date-bound, and you can drop old year partitions for archival instead of running expensive DELETE jobs.

MongoDB: Sharding is built-in and first-class. You define a shard key when creating a collection. Mongo's balancer automatically moves chunks between shards. For claims documents, `{ tenantId: 1, claimId: 1 }` as a compound shard key gives you tenant isolation and even distribution.

Kafka: Partitioning in Kafka is also hash-based on the message key. `claimId` as the Kafka message key ensures all events for the same claim go to the same partition — preserving order per claim. This is important when consumers process claim lifecycle events (SUBMITTED → VALIDATED → ADJUDICATED) and order matters.

Common Interview Q&A

Q: What's the difference between partitioning and sharding?

Partitioning splits data within a single DB server. Sharding splits data across multiple DB servers. Sharding is application-level (your code decides which server to query); partitioning is DB-engine-level (the engine handles it transparently).

Q: What is a hot partition and how do you fix it?

A hot partition is one that receives disproportionately more reads or writes than others — defeating the purpose of partitioning. Happens with range partitioning on date (current date always hot) or list partitioning on uneven categories. Fix: switch to hash partitioning for write distribution, add caching in front of hot reads, or use consistent hashing with virtual nodes.

Q: When would you NOT partition?

When your table is small (under a few million rows with good indexes, partitioning adds overhead with no benefit). When your queries don't align with the partition key (you'd scan all partitions anyway). When operational complexity isn't worth it for your scale.

Q: Cross-shard queries — how do you handle them?

Avoid them by design — choose a shard key that keeps related data together. When unavoidable (reporting, admin queries), use scatter-gather: query all shards in parallel, merge results in the application layer. Or maintain a separate read model (Elasticsearch, data warehouse) that aggregates across shards for reporting use cases.

6/6/2026, 6:51:40 AM

more system design Q&A for a senior engineer and principal engineer

6/8/2026, 12:04:30 PM

System Design Q&A — Senior & Principal Engineer Level

Category 1: Foundational Design Decisions

Q: How do you choose between SQL and NoSQL for a new service?

Don't start with the technology — start with the access patterns. Ask: what queries will this service run most often? If the answer involves joins across multiple entities, transactional consistency across rows, and a stable schema — SQL. If the answer involves fetching one document by ID, storing variable-shape data, or writing millions of events per second — NoSQL.

The second question is consistency requirement. Financial data, claims, inventory — strong consistency needed — SQL. User activity feeds, product catalogs, session data — eventual consistency acceptable — NoSQL.

Third question is scale trajectory. If you expect 100x write growth in 2 years and horizontal sharding is in your future, pick a DB that supports it natively (Cassandra, DynamoDB, MongoDB) rather than retrofitting sharding onto a relational DB.

In practice: most systems need both. Your Availity Claims module uses Aurora MySQL for transactional claim records and MongoDB for variable-schema claim documents — that's the right call.

Q: When would you use eventual consistency and how do you handle the consequences?

Choose eventual consistency when availability and partition tolerance matter more than instantaneous accuracy. User profile updates, social activity feeds, product view counts, recommendation engines — these are fine with data that's a few seconds stale.

The consequences you must handle explicitly:

Read-your-own-writes problem: User updates their profile, immediately reads it back, and sees the old value (read hit a replica that hasn't caught up). Fix: route reads to primary for a short window after a write, or use a version token that the client sends back.

Conflicting writes: Two nodes accept writes to the same record simultaneously. Fix with last-write-wins (LWW) using timestamps, or vector clocks to detect and resolve conflicts, or CRDTs (conflict-free replicated data types) for counters and sets.

Stale cache serving wrong data: Fix with event-driven invalidation — write to DB → publish event → cache listener deletes the key — rather than relying on TTL alone.

The interviewer wants to hear that you don't just choose eventual consistency and hope for the best. You choose it deliberately and handle the edge cases explicitly.

Q: How do you design a system to handle duplicate requests?

Idempotency is the answer — an operation executed multiple times produces the same result as executing it once.

At the API layer: accept an `Idempotency-Key` header (UUID generated by client). On first request, store `{key → response}` in Redis with a 24h TTL, process normally, return response. On retry with same key, return cached response immediately without reprocessing. This handles network timeouts where the client doesn't know if the request succeeded.

At the database layer: use unique constraints on natural business keys. `UNIQUE(claimId, payerId, serviceDate)` prevents duplicate claim records even if application-level deduplication fails.

At the Kafka consumer layer: maintain a processed event ID set in Redis. Before processing, check `SETNX eventId`. If key already exists, skip. If not, process and set the key. Redis TTL cleans up old keys.

The key principle: idempotency must be enforced at multiple layers because any single layer can fail.

Q: How do you design for zero-downtime deployments?

This requires thinking across four layers:

Database schema changes: Never make breaking schema changes in a single deployment. Use expand-contract pattern. Step 1 (expand): add new column as nullable, deploy new code that writes to both old and new columns. Step 2: backfill old rows. Step 3 (contract): remove old column after all services are on new code. A single migration that renames a column will break the running version of your service during the deployment window.

API versioning: Keep old API versions alive while new clients migrate. `/v1` and `/v2` both served simultaneously. Never change a live contract.

Feature flags: Deploy code dark (feature flag off). Enable for 1% of traffic. Ramp to 10%, 50%, 100%. Roll back instantly by flipping the flag — no deployment needed.

Kubernetes rolling updates: Deploy new pods one at a time. Health check passes before old pod is terminated. `maxUnavailable: 0, maxSurge: 1` ensures no capacity loss during rollout. Readiness probe gates traffic — new pod only receives requests once it's fully initialized.

Category 2: Scalability & Performance

Q: Walk me through how you'd handle a 100x traffic spike on a claims submission endpoint.

First, distinguish between a gradual ramp and a sudden spike. Gradual ramp → auto-scaling handles it. Sudden spike (say, open enrollment period — every American submits insurance claims the same week) needs pre-planned capacity.

Immediate layer (absorb the spike):
API Gateway rate limiting protects downstream services from being overwhelmed. Return `429` with `Retry-After` for excess requests rather than letting them cascade inward and crash the DB.

Application layer:
Kubernetes HPA scales pods based on CPU and custom metrics (queue depth). Pre-scale before known peak events — don't wait for HPA to react.

Async offloading:
Submission endpoint accepts the claim, writes to Kafka, returns `202 Accepted` with a tracking ID. Actual processing (eligibility check, adjudication) happens async. The HTTP

layer is now decoupled from processing capacity — you can absorb 100x submissions even if processing takes longer.

Database protection:
Write-through cache for lookup data (payer config, fee schedules). Connection pooler (PgBouncer) prevents connection exhaustion when pod count multiplies. Read replicas absorb any reads that can tolerate slight lag.

The honest answer to the interviewer: You also pre-load test at 2x, 5x, 10x, 50x expected peak, find the breaking point in staging, and fix it before it finds you in production.

Q: What is the N+1 query problem and how do you fix it at scale?

N+1 happens when you fetch N parent records and then issue one query per parent to fetch its children.

```
// N+1 — fetches 100 claims, then 100 separate queries for each claim's payer
List<Claim> claims = claimRepository.findByStatus("SUBMITTED"); // 1 query
claims.forEach(c -> c.getPayer().getName()); // N queries (lazy load)
```

At 10 records this is invisible. At 100,000 records this is 100,001 queries killing your DB.

Fix 1 — JOIN FETCH in JPQL:

```
@Query("SELECT c FROM Claim c JOIN FETCH c.payer WHERE c.status = :status")
List<Claim> findByStatusWithPayer(String status);
```

One query with a JOIN. Always.

Fix 2 — Batch fetching:
Hibernate `@BatchSize(size = 100)` — instead of N queries, issues `ceil(N/100)` queries using `WHERE payerId IN (...)`.

Fix 3 — DTO projections for read-only views:
Don't load full entities with all relationships when you only need 3 fields for a list view. Use

`SELECT new ClaimSummaryDTO(c.claimId, c.status, p.name) FROM Claim c JOIN c.payer p`.

At principal level: Instrument your persistence layer. Log query counts per request in non-production environments. Any request that issues more than 5 queries is suspicious and should be reviewed. Tools like Hibernate Statistics, p6spy, or Datadog APM show you query counts per endpoint automatically.

Q: How do you design a rate limiter?

A rate limiter controls how many requests a client can make in a time window. You need it at the API Gateway layer (per API key) and sometimes at the service layer (per user, per endpoint).

Algorithms:

Token bucket: Each client has a bucket with capacity N. Tokens refill at rate R per second. Each request consumes one token. If bucket is empty, reject. Allows bursts up to bucket capacity. Most natural for API rate limiting.

Sliding window log: Store timestamp of every request in Redis sorted set. On new request, remove entries older than window, count remaining, compare to limit. Precise but memory-heavy for high-volume clients.

Sliding window counter: Blend of fixed window and sliding. `current_count * ((window - time_since_last_window) / window) + previous_window_count`. Memory efficient, approximate but close enough.

Fixed window counter: Count requests per fixed time bucket (per minute). Simple. Problem: burst at window boundary — 100 requests in last second of minute 1 + 100 requests in first second of minute 2 = 200 in 2 seconds, but both windows show 100.

Implementation with Redis:

```
// Token bucket in Redis
MULTI
  GET user:123:tokens          // check current tokens
  GET user:123:last_refill     // time of last refill
EXEC
// calculate tokens to add based on elapsed time
// if tokens > 0: DECR and allow; else: reject with 429
```

Distributed rate limiting concern: In a multi-instance deployment, each instance has its own counter unless you centralize in Redis. Redis with atomic Lua scripts gives you exact counts across all instances. Lua script executes atomically — no race condition between check and decrement.

Q: How do you design for database connection pool exhaustion?

Connection pool exhaustion is one of the top causes of production outages under load. Every thread waiting for a DB connection is a thread not serving users.

Root causes:

- Pool too small for traffic volume
- Queries taking too long (holding connections longer than needed)
- Connection leak (connections not returned to pool — missing finally block, unclosed ResultSet)
- Sudden traffic spike multiplied by pod count exceeds DB max_connections

Design-level fixes:
Add a connection pooler (PgBouncer for PostgreSQL, ProxySQL for MySQL) between your services and the DB. PgBouncer maintains a small pool of actual DB connections and queues application requests. Your 50 pods each think they have 20 connections — PgBouncer multiplexes them into 100 real DB connections. DB sees 100 connections, not 1000.

Set `connection-timeout` in HikariCP (how long to wait for a pool connection before throwing). Without this, threads block indefinitely, thread pool exhausts, service appears hung.

Set `max-lifetime` and `keepalive-time` to avoid stale connections that the DB has closed on its side.

Monitor pool metrics: `hikaricp_connections_pending` > 0 for sustained period = you have exhaustion. Alert on this proactively.

Category 3: Distributed Systems

Q: Explain the Saga pattern and when you'd use it.

Sagas manage long-running distributed transactions across multiple microservices without using a two-phase commit (2PC). 2PC is avoided because it requires a distributed lock held across all services — if any service is slow or partitioned, everything blocks. Sagas replace this with a sequence of local transactions + compensating transactions.

Claim processing saga example:

```
Step 1: Claims Service    → create claim (PENDING)
Step 2: Eligibility Svc   → verify member coverage
Step 3: Auth Service      → get prior authorization
Step 4: Adjudication Svc  → calculate payment
Step 5: Payment Service   → disburse payment

If Step 3 fails:
  Compensate Step 2: release eligibility hold
  Compensate Step 1: mark claim REJECTED
```

Choreography saga: Each service publishes an event on success. The next service listens and acts. No central coordinator. Loose coupling. Hard to track overall saga state — you need to correlate events across services to understand where a saga is.

Orchestration saga: A central orchestrator (a Saga Orchestrator service or a workflow engine like AWS Step Functions, Temporal) sends commands to each service and handles failures. Easier to reason about, easier to monitor. Adds a central component that must be reliable.

When to use: Any business transaction that spans more than one service and needs rollback on failure. Claims adjudication, order fulfillment, financial transfers across accounts in different services.

Q: How does consistent hashing work and why does it matter?

Naive sharding: `shard = hash(key) % N`. Works fine until you add or remove a shard. Changing N from 4 to 5 changes the destination of nearly every key — massive data migration.

Consistent hashing maps both keys and servers onto a ring (0 to 2^32). A key is assigned to the first server clockwise from its position on the ring. When you add a server, only the keys between the new server and its predecessor move. When you remove a server, only that

server's keys move to its successor. On average, only `K/N` keys move (`K` = total keys, `N` = number of servers).

Virtual nodes: A single physical server maps to multiple points on the ring (virtual nodes). This prevents uneven distribution when server counts are small. If Server A has 3 virtual nodes and Server B has 3 virtual nodes, load is far more balanced than each having one position.

Where you see this: Cassandra uses consistent hashing for partitioning across nodes. Redis Cluster uses hash slots (16384 slots distributed across nodes — similar concept). DynamoDB uses it internally. Kafka doesn't — it uses simple hash modulo partition count, which is why repartitioning is painful.

Q: How do you handle distributed tracing in a microservices system?

When a request touches 6 services and fails in service 4, you need to reconstruct the full call chain — not just look at service 4's logs in isolation.

The mechanics: Every request gets a `traceId` at entry (API Gateway or first service). Every service call spawns a `spanId` (child of parent span). TraceId and spanId propagate through HTTP headers (`X-B3-TraceId`, `X-B3-SpanId` — B3 propagation format, or W3C `traceparent` header). Kafka message headers carry trace context across async boundaries.

Implementation in Spring Boot: Micrometer Tracing (formerly Spring Cloud Sleuth) auto-instruments RestTemplate, WebClient, and Kafka — no manual propagation code needed. Spans are exported to your tracing backend (AWS X-Ray, Jaeger, Zipkin, Datadog APM).

What it gives you: A waterfall view showing every service call, its duration, and where time was spent. A 4-second request immediately shows: 3ms Claims service logic + 3800ms Eligibility service HTTP call + 197ms DB write. The bottleneck is obvious without grepping logs across 6 systems.

At principal level: Enforce trace propagation in your platform SDK. If teams build their own HTTP clients without propagation, traces break at that service boundary. Provide a platform-level HTTP client wrapper that handles propagation automatically — teams use it without thinking about it.

Q: What is the Outbox Pattern and why is it important?

The dual-write problem: you update the DB and then publish a Kafka event. These are two separate operations. If the DB write succeeds but the Kafka publish fails (network blip, broker unavailable), your DB has the new state but no event was published — downstream services never know about it. If you reverse the order, the event fires but the DB write fails — event describes something that never happened.

The Outbox pattern solves this with a single atomic operation:

```
In same DB transaction:
  1. Write claim record to claims table
  2. Write event to outbox table (claimId, eventType, payload,
published=false)
  → Transaction commits atomically — either both happen or neither

Separate Outbox Processor (CDC or polling):
  3. Read unpublished rows from outbox table
  4. Publish to Kafka
  5. Mark rows published=true
```

The DB transaction ensures steps 1 and 2 are atomic. The outbox processor ensures eventual publication to Kafka — it can retry until Kafka is available.

CDC approach (better): Use Debezium to watch the outbox table's binlog. Every INSERT to the outbox table triggers a Debezium event that publishes to Kafka automatically. No polling, near real-time, no custom scheduler needed.

This is a principal-level pattern — knowing it, explaining it clearly, and knowing when it's necessary (any dual-write across a DB and a message broker) separates senior from principal.

Category 4: Principal Engineer Thinking

Q: How do you design a platform that 50 engineering teams will build on?

This shifts from designing a system to designing a system of systems. The concerns change completely.

Standardization vs autonomy tradeoff: Too much standardization → teams can't move fast, platform becomes a bottleneck. Too little → every team invents their own

authentication, logging, circuit breakers — inconsistency, security gaps, operational chaos. The answer is: standardize the cross-cutting concerns, give freedom on business logic.

What to standardize (platform layer):

- Authentication and authorization (one library, one token format, one JWKS endpoint)
- Observability (structured logging format, required MDC fields: traceId, tenantId, userId; standard metrics naming conventions)
- Service mesh (mTLS between services, circuit breakers, retry policies — enforced by Istio/Envoy sidecar without teams needing to implement them)
- CI/CD pipeline templates (golden path — teams customize within it, not around it)
- Base Docker images (hardened, patched, with your APM agent pre-installed)

How you enforce without being a bottleneck: Provide opinionated starters (Spring Boot starters that auto-configure your standard observability, auth, and error handling). Teams inherit it by adding a dependency. Non-compliance surfaces in security scans and runbook checklists — not code review by platform team.

API contracts as the integration surface: Services communicate via versioned APIs and event schemas (Avro/Protobuf with a Schema Registry). Platform owns the Schema Registry. Breaking schema changes require a migration plan. This is how 50 teams can evolve independently without breaking each other.

Q: How do you decide when to break a service out of a monolith?

Breaking out too early is as bad as breaking out too late. Most "microservices" failures happen because teams decomposed before understanding their domain boundaries.

Signals that a module should be its own service:

- It needs to scale independently (claims submission spikes but eligibility lookup is steady — split them if scale diverges)
- It has a different deployment lifecycle (the ML fraud scoring model deploys 10x per day; billing deploys monthly — different cadence is a reason to split)
- It has a different SLA (payment processing needs 99.99% uptime; document storage needs 99.9%)

- Team ownership is clear and stable (Conway's Law — your architecture will mirror your org chart; design the service boundary to match a team boundary)
- The data boundary is clean (the service can own its DB with no shared tables)

Signals NOT to split:

- "It's a big class" — size alone is not a reason
- No clear data ownership boundary — if two "services" need to join across each other's tables, they're not actually separate
- The team is 2 people — you'll spend more time on service infrastructure than product

The Strangler Fig pattern for incremental extraction: Don't rewrite the monolith. Put a proxy in front. Route new traffic for the target module to the new service. Old module still handles existing traffic. Gradually migrate. The monolith shrinks as the new services absorb its responsibilities. You never have a big bang cutover.

Q: How do you approach capacity planning?

Capacity planning at principal level is about avoiding surprises — both the "we ran out of capacity" surprise and the "we provisioned 10x what we needed and burned the budget" surprise.

Step 1 — Establish baselines:
Current RPS per endpoint, P99 latency, DB connection utilization, Kafka consumer lag, pod CPU/memory, cache hit rate. If you don't have these numbers today you're operating blind.

Step 2 — Model growth:
Business forecasts + historical growth rate → traffic projection for 6/12/18 months. For Availity: open enrollment periods are predictable spikes. Model them separately from baseline growth.

Step 3 — Load test to find limits:
Run load tests in staging at 2x, 5x, 10x current peak. Find where latency degrades, where errors start, what component breaks first. This is your headroom measurement.

Step 4 — Set alerts before limits:
If your DB connection pool exhausts at 90% utilization, alert at 70%. If pod CPU degrades at 80%, alert at 60% and auto-scale. Never find out you're at the limit by seeing errors in production.

Step 5 — Pre-scale for known events:
Open enrollment, marketing campaigns, batch job windows — pre-provision capacity 30 minutes before. Auto-scaling reacts; pre-scaling prevents the reaction lag from becoming an outage.

For the interview: Show that you think in SLOs, not just availability. "We target P99 < 500ms at peak load" is a capacity planning constraint, not just a monitoring metric.

Q: How would you design an event-driven architecture for a healthcare claims platform?

This is a design question with no single right answer — the interviewer wants to see your reasoning.

Core principle: Every state change in claims processing is an event. Model the business process as a series of domain events, not as a series of service calls.

```
ClaimSubmitted
    → EligibilityCheckRequested
    → EligibilityVerified / EligibilityDenied
    → AuthorizationRequested
    → AuthorizationApproved / AuthorizationDenied
    → ClaimAdjudicated
    → PaymentInitiated
    → PaymentCompleted / PaymentFailed
    → ClaimFinalSettled
    → EOBGenerated (Explanation of Benefits)
    → ProviderNotified
```

Event schema design: Every event has a standard envelope:

```
{
  "eventId": "uuid",
  "eventType": "ClaimSubmitted",
  "aggregateId": "claimId",
  "aggregateType": "Claim",
  "tenantId": "availity-tenant-001",
  "timestamp": "2026-06-05T10:00:00Z",
```

```
  "version": 1,
  "payload": { ... }
}
```

Schema Registry (Confluent or AWS Glue) enforces Avro/Protobuf schemas. Producers register schemas. Consumers validate on consumption. Breaking schema changes require new version — consumers opt in.

Event sourcing consideration: Store every event as the source of truth, not just the current state. The claim's current state is derived by replaying its event stream. Benefits: full audit trail (HIPAA requirement), time-travel queries ("what was the claim state on date X?"), ability to add new projections without touching historical data. Cost: query complexity, snapshot management for long-lived aggregates.

Failure handling in event-driven systems:
Dead Letter Topics for unprocessable events. Circuit breakers on consumer processing (if downstream DB is down, pause consumption rather than losing events). Idempotent consumers (event replayed from DLT must not cause duplicate processing). Saga orchestrator for multi-step flows with compensating transactions.

Principal-level concern: Governance. With 20 services producing events, schema drift and undocumented event contracts become a serious problem. Invest in an event catalog (Backstage, AsyncAPI spec) — every event type documented, every producer and consumer registered. Without this, event-driven architecture becomes an undocumented spaghetti of async dependencies.

Category 5: Real Production Scenarios

Q: Your service's P99 latency suddenly spiked from 200ms to 5 seconds. Walk me through your diagnosis.

Start with what changed. Deployment in the last hour? Config change? Traffic spike? Cron job that fired? Correlation with an external event is the fastest path to root cause.

Check the 4 golden signals simultaneously — not sequentially. Latency up (confirmed). Traffic: normal, higher, or lower? If traffic is normal but latency is up, it's not a load problem — it's a quality problem. Errors: are 5xx errors accompanying the latency? If yes, something is failing. If no, something is slow but succeeding. Saturation: CPU, memory, DB connections, thread pool queue depth — which resource is at ceiling?

Pull distributed traces for slow requests. The trace waterfall shows exactly which span is taking 4.8 of the 5 seconds. If it's a DB span, go to slow query log. If it's an external HTTP call, check that service's health. If it's within your service, check thread pool metrics and GC logs.

Common culprits in this order: unindexed query that was fast at low data volume but degrades as table grows past a threshold; external service degradation (payer eligibility API is slow, your circuit breaker isn't tripping because it's slow not failing); GC pressure from memory leak — heap growing, GC pausing more frequently, each pause adds latency; connection pool contention — threads waiting for a DB connection because a previous slow query holds connections longer.

Fix, deploy, verify in metrics — not just "it looks better." Confirm P99 returned to baseline and write a postmortem.

Q: How would you handle a situation where two services are causing a deadlock in the database?

Database deadlocks happen when Service A holds lock on Row 1 and waits for Row 2, while Service B holds lock on Row 2 and waits for Row 1. The DB detects the cycle and kills one transaction.

Immediate mitigation: The DB will surface deadlock errors in logs (`Deadlock found when trying to get lock`). The killed transaction gets an exception — it should retry. Implement retry with exponential backoff on deadlock exceptions specifically.

Root cause analysis: Enable DB deadlock logging (`innodb_print_all_deadlocks = ON` in MySQL). The log shows exact transactions, exact rows locked, exact lock order. This reveals the pattern.

Fix — enforce consistent lock ordering: If two transactions both need to lock ClaimRow and PayerRow, always lock them in the same order (ClaimRow first, then PayerRow) in both transactions. Deadlocks occur when two transactions acquire locks in opposite orders. Enforcing a global lock ordering by primary key prevents deadlock cycles.

Fix — reduce lock scope: Use `SELECT ... FOR UPDATE` only where truly needed. Use optimistic locking (`@Version` in JPA) instead of pessimistic locking — optimistic locking doesn't hold DB locks, it retries on conflict. For claims processing, optimistic locking is usually sufficient and eliminates a whole class of deadlock scenarios.

Fix — shorter transactions: Long transactions hold locks longer, increasing collision probability. Move non-DB work (HTTP calls, calculations) outside the transaction boundary.

Q: How do you design a multi-tenant system where tenant data must be strictly isolated?

Three models with different isolation levels and operational tradeoffs:

Silo model (separate DB per tenant): Each tenant has their own DB instance. Absolute data isolation. Easy to meet regulatory requirements (HIPAA, data residency). Simple to delete a tenant's data — drop the DB. Cost: operational overhead scales with tenant count. 1000 tenants = 1000 DBs to manage, monitor, patch.

Bridge model (separate schema per tenant, shared DB server): One DB server, one schema per tenant (`availity_tenant_001.claims`, `availity_tenant_002.claims`). Good isolation, lower cost than silo. Row-level security policies still needed to prevent accidental cross-schema queries. Works well for tens or hundreds of tenants.

Pool model (shared tables, tenantId column): All tenants share tables. `tenantId` on every table, every query. Lowest cost, simplest operations. Isolation is purely logical — a bug in your code (missing WHERE tenantId = ?) leaks data across tenants. Requires Row Level Security at the DB layer (PostgreSQL RLS, or a Hibernate filter that injects tenantId on every query automatically).

Hybrid: Large enterprise tenants get silo or bridge. Small tenants share the pool model. This is what most mature SaaS platforms do.

For Availity specifically: Healthcare data isolation is a HIPAA requirement. The bridge or silo model is appropriate for large health system clients. Pool model with RLS is acceptable for smaller provider groups. The key engineering investment is a tenant context that propagates through every layer — HTTP request → Spring security context → JPA filter → Kafka message header → Redis key prefix. Every data access is automatically scoped.

6/8/2026, 12:07:16 PM

is this enough

6/8/2026, 12:08:07 PM

Honestly — for a **Senior Engineer** level: yes, more than enough.

For **Principal Engineer** level: the concepts are covered, but concept knowledge alone won't get you there. Here's an honest assessment:

What You Have — Strong Coverage

- System design fundamentals (LB, Cache, DB, MQ, Gateway, Microservices)
- Distributed systems patterns (Saga, Outbox, Consistent Hashing, Circuit Breaker)
- Production thinking (bottleneck analysis, fault tolerance, debugging approach)
- DB partitioning in depth
- Performance tuning
- Senior + Principal Q&A

This covers roughly **80% of what gets asked** at senior level interviews.

What's Still Missing

For Senior Level (smaller gaps)

- **Designing specific systems end-to-end** — URL shortener, notification system, rate limiter, leaderboard, chat system. Interviewers at senior level often give you 45 minutes to design one system from scratch. Reading concepts isn't the same as practicing the full design flow.
- **Estimation / back-of-envelope math** — "How many servers do you need? How much storage?" Interviewers test if you can reason numerically. Example: *1M claims/day × 10KB average payload = 10GB/day = 3.6TB/year. Need partitioning strategy.*
- **Security basics** — OAuth2 flow, JWT validation, mTLS between services, secrets management. At least one question touches this at senior level.

For Principal Level (larger gaps)

Principal interviews test a different thing entirely. It's less "do you know the pattern" and more **"how do you make decisions under ambiguity and influence technical direction."**

What you still need:

- **Tech debt prioritization** — how do you decide what to rewrite vs leave alone? How do you convince leadership to invest in platform work?
- **Engineering org design** — team topologies, platform teams vs stream-aligned teams, how Conway's Law affects your architecture decisions
- **Build vs buy decisions** — when to use managed services vs build in-house, how to evaluate vendor risk
- **Incident command thinking** — how do you lead a P0 incident with 5 teams involved? Communication structure, war room protocol, blameless postmortem culture
- **Long-horizon thinking** — "What does this system look like in 3 years? What decisions today make that harder or easier?"

The Biggest Real Gap

You can read all of this and still struggle in interviews.

The gap between reading system design notes and performing well in a 45-minute live design session is significant. What trips people up isn't knowledge — it's:

- Starting too detailed too fast (jumping to DB schema before clarifying requirements)
- Not driving the conversation (waiting for the interviewer to ask rather than leading)
- Not making explicit trade-off statements ("I'm choosing X over Y because of Z, the downside is W")
- Going silent when challenged instead of thinking out loud

Honest Recommendation

Goal	What to do next
Senior interview in 2–3 weeks	Practice 5–8 full system designs end-to-end out loud. Do estimation. You're ready on knowledge.

Goal	What to do next
Principal interview	Add the gaps above + practice leading design discussions, not just answering them
Both	Time-box reading. Spend 70% of remaining prep time doing, not reading

The one thing that moves the needle most: Pick a system (design Twitter feed, design a payment system, design claims processing from scratch) and talk through the full design for 45 minutes — requirements → estimation → HLD → deep dive → failure modes — without looking at notes.

That's the real test. Do you want to practice one end-to-end right now?

6/8/2026, 12:08:31 PM

give more Q&A for principal engineer

6/8/2026, 12:09:07 PM

Principal Engineer System Design Q&A

Category 1: Architecture Decision Making

Q: How do you decide whether to build a shared platform service or let each team solve it independently?

This is one of the most consequential decisions a principal engineer makes — and getting it wrong in either direction is expensive.

Build a shared platform service when:

- The problem is identical across teams (authentication, logging, rate limiting, feature flags). If every team is solving the same problem, you're paying N times for one solution and getting N slightly different implementations with N different failure modes.
- The cost of inconsistency is high. Security is the clearest example — if 3 teams implement JWT validation slightly differently, one of them will get it wrong and you have a breach.
- The domain knowledge required is deep and specialized. Building a reliable distributed rate limiter or a HIPAA-compliant audit log requires expertise most product teams don't have and shouldn't need to develop.

Let teams solve it independently when:

- Requirements actually differ across teams more than they appear to. Premature abstraction of something that looks common but has subtle domain-specific differences creates a platform that serves nobody well.

- The shared service becomes a bottleneck. If every team needs a change to the platform and the platform team has a 6-week queue, you've optimized for consistency and destroyed velocity.
- The blast radius of the shared service failing is unacceptable. A shared authentication service going down takes every team with it. Sometimes the right answer is "each team runs their own instance of this library."

The framework I use: Ask three questions. One — is the problem truly identical or just superficially similar? Two — what is the cost of the platform team becoming a dependency? Three — what is the cost of divergent implementations over 2 years? If the answer to three is "serious security or reliability risk," build the platform. If the answer to two is "teams will slow down significantly," invest in making the platform team faster before centralizing more.

At Availity relevance: Claims, Eligibility, and Auth teams all need audit logging for HIPAA compliance. That's a platform service — the compliance requirement is identical, the cost of divergent implementations is regulatory risk. But claims processing workflow logic is domain-specific — a shared "claims processing framework" would be premature abstraction that creates coupling without real benefit.

Q: How do you evaluate whether a system design is good?

Most engineers evaluate designs by asking "does it work?" Principal engineers evaluate designs on five dimensions simultaneously.

Correctness: Does it actually solve the stated problem? Does it handle the edge cases — what happens when the third-party service is down, when the DB is full, when a message is processed twice? A design that works on the happy path but has no answer for failures is incomplete.

Simplicity: Is there a simpler design that meets the requirements? Complexity is a liability — every moving part is a thing that can fail, a thing someone has to understand, a thing that needs to be monitored. The best designs feel obvious in retrospect. If you need a 30-minute explanation to describe your architecture to a new engineer, it's too complex.

Operability: Can you run this in production at 3am when something goes wrong? Does it have health checks? Are its failure modes observable? Can you deploy it without downtime? Can you roll it back? A design that works perfectly but is impossible to debug in production is a bad design.

Evolvability: How hard is it to change this in 18 months when requirements shift? Are the boundaries clean? Is the schema flexible enough? Is there tight coupling that makes a small change require touching 6 services? Good designs are easy to change — bad designs become legacy systems that nobody wants to touch.

Cost: Does the operational and infrastructure cost match the business value? A design that requires 20 engineers to operate and $500k/month in infrastructure for a feature used by 100 customers is a bad design regardless of its technical elegance.

The question I ask at the end of every design review: "If this system is successful and traffic grows 10x in 18 months, which part of this design will we regret?" That question surfaces assumptions baked in that will become constraints later.

Q: When should you rewrite a system vs incrementally improve it?

The pressure to rewrite is almost always wrong, and almost always comes from engineers who didn't build the original system and don't understand why it is the way it is.

The case against rewrites:
The original system has years of edge case handling baked in — bugs that were fixed, business rules that were added because of real incidents, workarounds for third-party API quirks. None of this is in the code comments. A rewrite throws all of it away and spends 18 months rediscovering it. Joel Spolsky called this "the single worst strategic mistake a software company can make." The rewrite is always estimated at 6 months and takes 3 years, and on day one of the new system you start getting bug reports about things the old system handled correctly.

When rewrite is genuinely justified:
The technology has become a hard constraint — the language runtime has no security patches, the framework is unmaintained, you literally cannot hire people who know it. The architecture prevents a business requirement that cannot be worked around — you need horizontal scaling and the system is fundamentally stateful in a way that can't be extracted. The cost of maintaining the old system is measurably higher than the cost of replacement — you can quantify this, not just feel it.

The incremental alternative (almost always right):
Strangler Fig — proxy in front of the old system, new service handles new traffic, old system handles old traffic, gradually migrate. You never have a big bang cutover. The old system stays in production until every function has been migrated and verified. You can stop at any point. Risk is contained.

Event interception — publish events from the old system (via CDC on its DB or via a thin event-publishing wrapper) and build new services that react to those events. New capabilities live in new services. Old system is unchanged.

The honest answer for interviews: "I would want to understand what specific problems we're solving and whether a rewrite is the only path to solving them. In my experience, the answer is usually no, and the Strangler Fig pattern lets us improve the system incrementally while keeping it in production."

Q: How do you handle technical debt at scale across multiple teams?

Technical debt at scale is a portfolio management problem, not a code quality problem. You can't fix all of it. You have to make deliberate choices about which debt to pay, which to live with, and which to stop accumulating.

Classify debt before you address it:
Reckless debt — shortcuts taken under pressure with full knowledge that they'd cause problems. This is the most urgent. It's actively costing you today in incidents, slow deployments, and developer frustration.
Prudent debt — deliberate decisions to ship faster with a known tradeoff. "We'll use a simple synchronous flow now and move to async when we need the scale." This is fine as long as the tradeoff was conscious and the scale signal exists to trigger the investment.
Inadvertent debt — design decisions that were correct at the time but are now wrong because the system evolved. The data model was designed for single-tenant and now you need multi-tenant. This is unavoidable — systems evolve.

How to prioritize:
Debt that is blocking a strategic business goal gets addressed first — this is the only way to get engineering investment approved by leadership. "We can't launch the new payer integration because the auth service can't support the new OAuth flow" — that debt gets paid.
Debt that is causing recurring incidents gets addressed second — calculate the cost in engineer-hours per quarter spent on incidents caused by this component. Present that number. It makes the ROI case automatically.
Debt that is slowing down feature development gets addressed third — measure it. If adding a feature to the claims module takes 3 weeks because the module has no test coverage and the data model is tangled, that's a measurable velocity cost.

The structural fix: Allocate a non-negotiable percentage of every sprint to debt reduction — 20% is a common target. Don't let this be negotiated away when roadmap pressure increases. The teams that consistently skip debt reduction are the ones with the most incidents 18 months later.

Principal-level responsibility: Prevent new debt from being created at the same rate you're paying it down. Enforce architecture decision records (ADRs) for significant decisions. Make tech debt visible in your architecture review process — new design proposals that introduce known debt patterns get flagged before they're built, not 2 years later.

Category 2: Distributed Systems Deep Cuts

Q: Explain the differences between at-most-once, at-least-once, and exactly-once delivery. When do you use each?

These describe what guarantee a messaging system makes about whether a message gets delivered to the consumer.

At-most-once: Message is sent once. If it's lost (broker crash, network drop), it's lost. No retry. Consumer processes it 0 or 1 times. Use when: losing some messages is acceptable and duplicate processing is more harmful than loss. Metrics, analytics events, log aggregation — losing 0.01% of events doesn't matter. Never use for financial transactions or claims.

At-least-once: Message is retried until acknowledged. Consumer may receive it multiple times. Use when: you cannot afford to lose messages, and your consumer is idempotent (processing the same message twice is safe). This is the right default for most systems. Kafka's default behavior — producer retries on failure, consumer reprocesses from last committed offset on restart.

Exactly-once: Message is delivered and processed exactly once, even in the presence of failures. This is the hardest guarantee to provide. Kafka implements it via idempotent producers (each message has a sequence number, broker deduplicates) + transactional producers (atomic write across multiple partitions) + read-process-write within a Kafka transaction. Outside of Kafka-to-Kafka flows, true exactly-once is almost impossible — as soon as you involve an external DB or API, you're back to at-least-once with idempotent consumer logic.

The practical reality: Build at-least-once delivery + idempotent consumers. This is achievable, well-understood, and gives you the same effective result as exactly-once for most use cases without the complexity and performance overhead of distributed transactions.

For claims processing: Claim submitted event must be at-least-once — you cannot lose a claim. Consumer (eligibility check, adjudication) must be idempotent — processing ClaimSubmitted twice must not create two eligibility checks or two payments. Use the Outbox pattern for reliable at-least-once, and event ID deduplication in the consumer for idempotency.

Q: How does leader election work in distributed systems and when do you need it?

Leader election is the process by which a group of nodes selects one node as the coordinator for a task. You need it when a task must be performed by exactly one node — running a scheduled job, being the primary writer in a replication setup, coordinating a distributed lock.

Why you can't just pick a leader statically: Static leaders become single points of failure. If the designated leader crashes, nothing works until a human intervenes. You need automatic failover.

How it works (Raft algorithm — the most understandable):
Nodes start as followers. If a follower doesn't hear from a leader for an election timeout (randomized, 150–300ms), it becomes a candidate and requests votes. A node votes for a candidate if it hasn't voted in this term and the candidate's log is at least as up-to-date. A candidate that gets votes from a majority becomes leader. The majority requirement prevents split-brain — two nodes both thinking they're leader simultaneously.

ZooKeeper approach: Nodes create ephemeral sequential znodes. The node that created the lowest-numbered znode is leader. If the leader crashes, its ephemeral znode disappears (ZooKeeper session expiry) and the next node becomes leader. ZooKeeper itself uses ZAB (ZooKeeper Atomic Broadcast) which is similar to Raft.

etcd (Kubernetes uses this): Raft-based. Kubernetes uses etcd for all cluster state and leader election for controller manager, scheduler. `kubectl get leases -n kube-system` shows current leaders.

When you need it in application design:
Scheduled jobs — you have 10 pods but the nightly reconciliation job should run once. Use

a distributed lock or leader election via Redis `SET NX` with TTL or a Kubernetes Lease object. Without it, all 10 pods run the job simultaneously.
Cache warming — one pod should warm the cache on startup, not all 10 simultaneously hammering the DB.
Kafka partition assignment — Kafka's group coordinator does leader election for consumer group rebalancing.

Q: How do you design for data consistency across microservices without distributed transactions?

Distributed transactions (2PC) are theoretically correct but practically dangerous — they hold locks across service boundaries, are slow, and if the coordinator crashes mid-transaction you're left in an indeterminate state. The industry has largely moved away from them.

The core insight: You don't need distributed transactions if you design your service boundaries correctly. If two operations must be atomic, they should be in the same service with the same DB. If they're genuinely in different services, accept that consistency will be eventual and design for it.

Pattern 1 — Outbox + CDC:
Atomic write to DB + outbox table in one local transaction. Debezium publishes outbox events to Kafka. Downstream services update their own state. Eventual consistency — the downstream service will be consistent, just not instantly. Good for: claims state propagation, audit trail, notification triggers.

Pattern 2 — Saga:
Each service does its local transaction and publishes an event. Next service picks up the event and does its local transaction. Compensating transactions for rollback. Good for: multi-step business workflows where each step has a clear compensating action.

Pattern 3 — Event sourcing as single source of truth:
The event log is the source of truth. All services derive their state by consuming the event log. Consistency comes from all services processing the same ordered event log. Good for: audit-heavy domains, HIPAA compliance requirements, time-travel queries.

Pattern 4 — Synchronous calls with idempotency:
Service A calls Service B synchronously. If B fails, A retries. B must be idempotent. This is the simplest pattern and is underused — people reach for Kafka when a synchronous

idempotent call would work fine. Good for: low-volume, latency-sensitive operations where you can afford to wait for confirmation.

The question that determines which pattern: "Can this business operation tolerate eventual consistency, or does the user/system need to see the consistent state immediately?" Claims adjudication — eventual is fine, the provider doesn't need real-time confirmation of payment calculation. Payment initiation — you need to know it succeeded before telling the user.

Q: What is backpressure and how do you implement it?

Backpressure is a mechanism for a downstream system to signal to an upstream system to slow down production when it can't keep up with consumption. Without it, a fast producer overwhelms a slow consumer — queue grows unbounded, memory exhausts, system crashes.

The problem without backpressure:
Claims submission API receives 10,000 claims/second. Eligibility service can process 1,000/second. Without backpressure, Kafka topic lag grows by 9,000/second, consumer memory pressure builds, eventually the consumer falls over, lag grows faster, you have a cascading failure.

Backpressure in Kafka:
Kafka naturally provides backpressure via consumer lag visibility. The mechanism is: monitor consumer lag. When lag crosses a threshold, scale out consumers (HPA triggered by custom metric from Kafka exporter). If you can't scale fast enough, rate limit the producer — return 429 at the API layer, or use Kafka quotas to throttle producers per client ID.

Backpressure in reactive systems (Project Reactor / RxJava):
Reactive streams specification builds backpressure in at the API level. The subscriber signals demand upstream — "I can handle 100 items." The publisher only sends 100. If subscriber slows down, publisher slows down. `Flux.range(1, 1_000_000).limitRate(100)` — processes 100 at a time, requests more only when ready.

Backpressure in thread pool design:
Bounded queue in front of thread pool. When queue is full, reject new tasks with a defined rejection policy — either return error to caller (fail fast), run in caller's thread (slow down

the submitter), or drop and log. Never use unbounded queues — they hide the problem until OOM.

The principal-level point: Backpressure must be a conscious design decision at every integration boundary. Where does fast-producer-slow-consumer happen in your system? What is the signal that tells the producer to slow down? What happens when the queue is full? Answering these questions for every async boundary in your design is what separates a resilient system from one that fails spectacularly under load.

Category 3: Organizational & Leadership Thinking

Q: How do you drive adoption of a technical standard across teams that don't report to you?

This is fundamentally an influence problem, not a technical problem. Principal engineers operate with authority of expertise and trust, not organizational authority.

Start with understanding resistance:
Teams resist standards for legitimate reasons — the standard doesn't fit their use case, the migration cost is high, they've invested in a different approach and are being asked to throw it away. Understand the objection before trying to overcome it. If three teams say "this doesn't work for us" and you dismiss it as resistance to change, you're missing signal that your standard is wrong.

Make the standard easy to adopt:
The best standards are adopted because they make engineers' lives easier, not because they're mandated. Provide a Spring Boot starter that auto-configures your observability standard. Provide a Terraform module that sets up your standard VPC layout. Provide a GitHub Actions template for your standard CI pipeline. If adopting the standard is a one-line dependency change, teams adopt it. If it requires a 2-week migration, they don't.

Find champions in each team:
Identify the senior engineers in other teams who care about the problem the standard solves. Work with them to shape the standard — their input makes it better and creates advocates. When they tell their team "we helped design this and it solves a real problem," adoption follows.

Show don't tell:
Implement the standard in one system end to end. Show the metrics — adoption reduced

our incident response time by X, it reduced our security review time by Y. Concrete outcomes are more persuasive than architectural arguments.

When to escalate to mandate:
Security and compliance standards sometimes cannot be voluntary. HIPAA audit logging, encryption at rest, secrets management — these are mandates. Be clear about which standards are voluntary (good practices you're recommending) and which are non-negotiable (compliance requirements). Conflating them makes engineers distrust your judgment on both.

Q: How do you evaluate a system design proposed by a junior engineer and give constructive feedback?

The goal is to make the engineer better, not to demonstrate that you can find problems. How you give feedback matters as much as what feedback you give.

First — understand before evaluating:
Ask clarifying questions before identifying problems. "What constraints were you optimizing for?" "What alternatives did you consider?" "What are you most uncertain about?" Often what looks like a naive design choice was actually a deliberate tradeoff. If you critique it without understanding the reasoning, you'll look like you're not listening and the engineer will stop sharing work with you.

Identify the strongest parts first — genuinely:
Not as a softening tactic but because it calibrates the conversation. "The event-driven approach here is the right call for this scale, and the schema design is clean." Now the engineer knows what to keep. Feedback that's 100% critical leaves the engineer not knowing what they got right.

Prioritize feedback by impact:
Not every problem is equally important. "This API naming is inconsistent" and "this design has no failure handling for the payment step" are not equally critical. Give the design-correctness and reliability feedback first and most thoroughly. Note the style and naming issues briefly. Don't let minor issues crowd out major ones.

Ask questions instead of making statements where possible:
"What happens to the claim if the eligibility service is down when this runs?" is more effective than "You didn't handle eligibility service downtime." The first approach makes the engineer discover the gap — that learning sticks. The second approach just gives them an answer.

Distinguish between "this is wrong" and "this is a tradeoff":
"You need error handling here" is a correction. "I'd have chosen async over sync here, but your sync approach works at this scale" is a tradeoff discussion. Be clear about which you're giving. Junior engineers often can't tell the difference and treat all feedback as equally authoritative.

Q: A critical production service is down and it's affecting revenue. Walk me through how you lead the incident.

Incident leadership is a skill separate from technical skill. The instinct of technical people is to start debugging immediately — the right behavior is to organize the response first.

First 5 minutes — establish structure:
Declare the incident formally (PagerDuty, Slack incident channel). Assign roles explicitly — incident commander (owns communication and coordination, not debugging), technical lead (owns the investigation), communications lead (owns stakeholder updates). If you're the senior technical person, take technical lead. Assign someone else to incident commander if possible — you can't debug and coordinate simultaneously.

First 15 minutes — establish facts:
What is the user-visible impact? What is the scope (all users, specific tenants, specific operations)? When did it start? What changed in the last 2 hours (deployments, config changes, traffic patterns)? Post these facts in the incident channel. Everyone now has the same baseline — no parallel investigations based on different assumptions.

Investigation — structured hypothesis testing:
The worst incident behavior is 8 engineers all trying different things simultaneously with no coordination. State a hypothesis. Assign one person to test it. Results come back to the channel. Update or discard the hypothesis. Move to the next. This is slower per hypothesis but faster to resolution because you're not duplicating work.

Communication cadence:
Stakeholder updates every 15 minutes regardless of whether you have new information. "We are investigating, impact is X, no resolution yet" is a valid update. Silence breeds escalation. The update doesn't need to have the answer — it needs to exist.

Mitigation before root cause:
Your job during the incident is to restore service, not to understand why. If rolling back the deployment stops the bleeding, roll back now. Understand why later. "We're going to keep

it down until we understand the root cause" is almost never the right call when revenue is impacted.

Post-incident:
Blameless postmortem within 48 hours. Timeline, root cause, impact, what went well in the response, what didn't. Action items with owners and deadlines — not vague recommendations. The postmortem is where learning happens. The quality of your postmortems determines whether you have the same incident twice.

Q: How do you make a build vs buy decision for a new infrastructure component?

Build vs buy sounds like a technical decision but it's actually a business decision with technical inputs.

Start with what you're actually buying:
"Buy" in modern infrastructure usually means managed cloud services (AWS RDS vs self-managed MySQL, AWS MSK vs self-managed Kafka, Datadog vs self-managed Prometheus/Grafana) or SaaS products. The question is whether the value of the managed service justifies the cost and the vendor dependency.

The cost of build is almost always underestimated:
Building a distributed rate limiter sounds like a weekend project. Operating it in production — monitoring, scaling, handling edge cases, keeping it updated, oncall rotation — is a year of engineer time and counting. The build cost is not the implementation cost, it's the total cost of ownership over 3 years including the opportunity cost of not building product features instead.

When to buy:
The problem is not your core competency and not a competitive differentiator. AWS RDS, Kafka on Confluent, Datadog, PagerDuty — nobody wins in the market because they built a better MySQL. The managed service has operational characteristics (availability, security, compliance certifications) that would take years to match. You need it working in weeks, not months.

When to build:
The off-the-shelf solution doesn't fit your requirements and customization is impossible or as expensive as building. Your scale exceeds what any vendor can provide cost-effectively — at extreme scale, AWS's data transfer costs alone justify building your own CDN (Netflix did this). The component is genuinely a competitive differentiator — Uber's dispatch

algorithm, Google's search ranking. You have the expertise in-house to build and operate it.

The vendor dependency risk:
Factor in lock-in cost explicitly. If you build on AWS Step Functions for your Saga orchestrator, migrating away from AWS is now harder. If Confluent doubles their prices in year 3, what's your exit path? Good buy decisions include a clear answer to "what do we do if this vendor becomes unusable in 2 years?"

For Availity specifically: HIPAA compliance certification is a significant factor. A managed service with an existing BAA (Business Associate Agreement) and SOC2 compliance is worth a premium over building your own because the compliance cost of the self-managed alternative is enormous.

Category 4: System Design at Principal Scale

Q: Design a system that processes 1 million healthcare claims per day with sub-second API response and full audit trail.

Requirements clarification first:
Sub-second for what operation — submission acknowledgment or full adjudication result? At principal level you ask this because the answer completely changes the design. Assume: submission API returns sub-second acknowledgment, full processing is async within 4 hours, audit trail means every state transition is immutable and queryable.

Estimation:
1M claims/day = ~12 claims/second average. Open enrollment peak = 10x = 120 claims/second. Average claim payload = 10KB. Storage = 10KB × 1M × 365 = 3.65TB/year raw. Audit events = ~10 events per claim = 10M events/day.

High Level Design:

```
Client → API Gateway (Tyk/Kong)
       → Claims Submission Service (Spring Boot, 10 pods)
       → Kafka (ClaimSubmitted topic, 12 partitions)
       → Return 202 Accepted + claimId immediately

Kafka consumers (async pipeline):
ClaimSubmitted
```

```
  → Validation Service      → ClaimValidated / ClaimRejected
  → Eligibility Service     → EligibilityVerified /
EligibilityDenied
  → Adjudication Service    → ClaimAdjudicated
  → Payment Service         → PaymentInitiated
  → Notification Service    → ProviderNotified

Storage:
  Aurora MySQL (write model) — claim lifecycle state
  MongoDB (document store)  — full claim payload
  Elasticsearch             — audit event search
  S3                        — raw claim archive, cold storage
  Redis                     — idempotency keys, rate limiting
```

Audit trail design:

Every service publishes state transition events to an `audit-events` Kafka topic. A dedicated Audit Service consumes all events and writes to Elasticsearch (queryable) and S3 (immutable archive). Events are append-only — no updates or deletes. Event schema includes: eventId, claimId, eventType, previousState, newState, actorId, timestamp, serviceVersion. This satisfies HIPAA requirement for complete audit trail of PHI access and modification.

Sub-second submission:

Submission service receives claim → validates schema only (not business rules) → writes to outbox table in Aurora → publishes ClaimSubmitted to Kafka → returns 202 with claimId and estimated processing time. All in under 200ms. Business validation (eligibility, authorization) is async.

Idempotency:

Submission endpoint requires `Idempotency-Key` header. Redis stores `{key → claimId}` with 24h TTL. Duplicate submissions within 24h return the original claimId without creating a new claim.

Failure handling:

Each Kafka consumer has a Dead Letter Topic. Failed claims are routed to DLT with failure reason. A separate DLT processor retries with backoff or routes to manual review queue. No claim is silently lost.

Principal-level addition: Multi-tenancy. Every table has `tenantId`. Hibernate filter injects `tenantId` on every query automatically. Kafka message headers carry `tenantId` for consumer routing. Redis keys are prefixed with `tenantId`. Data isolation is enforced at every layer, not just the API boundary.

Q: How would you design the observability platform for a 200-service microservices system?

At 200 services, observability is not a tool choice — it's an engineering platform problem. The question is how you make observability effortless for 50 engineering teams without requiring each team to become a monitoring expert.

The three pillars and their implementation at scale:

Logs:
Standardize structured JSON logging across all services — enforce via a shared logging library that teams include as a dependency. Required fields: `traceId`, `spanId`, `tenantId`, `userId`, `serviceVersion`, `environment`, `severity`. Teams add their own fields on top.
Collection: Fluent Bit as a Kubernetes DaemonSet on every node, shipping to OpenSearch/Elasticsearch. Log retention: hot (7 days, fast search), warm (30 days, slower), cold (1 year on S3 Glacier for compliance).
Problem at 200 services: log volume is enormous and expensive. Implement sampling for debug/info logs in production. Always log warn/error. Never sample. Dynamic log level adjustment via a feature flag — turn on debug for one service for 15 minutes to diagnose, auto-reverts.

Metrics:
Prometheus for collection, Grafana for visualization. At 200 services with Kubernetes, use the Prometheus Operator — each service exposes `/actuator/prometheus` and Prometheus auto-discovers them via ServiceMonitor custom resources.
Standardize your metric naming convention: `{service}_{operation}_{unit}`. Example: `claims_submission_duration_seconds`, `eligibility_check_errors_total`. Without this, 200 services produce 200 different naming patterns and dashboards can't be templatized.
Provide a Grafana dashboard template that works for any service following the naming convention. Teams get a working dashboard on day one without building it.
Alert on SLOs not symptoms. Not "CPU > 80%" but "error rate > 0.1% sustained for 5 minutes" and "P99 latency > 1s sustained for 5 minutes."

Traces:
OpenTelemetry as the standard — vendor-neutral, works with Jaeger, Zipkin, Datadog, AWS X-Ray. Provide an OTel Java agent that teams include in their Docker base image. Zero code change for automatic instrumentation of Spring Boot, JDBC, Kafka, HTTP clients.
Sampling strategy: head-based sampling at 10% for normal traffic, tail-based sampling that captures 100% of traces with errors or latency > 1s. This gives you complete visibility on problems without storing every trace.
Trace propagation across Kafka: include traceId and spanId in Kafka message headers. Your OTel instrumentation should handle this automatically — verify it does before mandating the standard.

The platform team's responsibility:
Provide runbooks, not just tools. When an alert fires, the runbook should tell the on-call engineer where to look first, what queries to run, what the common causes are. A well-written runbook reduces mean time to resolution more than any tool upgrade.
Service Level Objectives as code — teams define their SLOs in a YAML file in their repo. The platform converts this to Prometheus recording rules and Grafana SLO dashboards automatically. SLO compliance becomes visible to both engineering and leadership.

Q: How do you design a multi-region active-active system?

Multi-region active-active means multiple regions each accept writes simultaneously and replicate to each other. This is the hardest distributed systems problem in practice because you're deliberately accepting write conflicts.

Why do it:
Latency — users in India get served from Mumbai, users in US get served from us-east.
Availability — a full AWS region outage (rare but happens) doesn't take down your system.
Regulatory — data residency requirements may mandate that patient data stays in-region.

The fundamental conflict problem:
Two users in different regions update the same claim simultaneously. Region A writes status = APPROVED. Region B writes status = REJECTED. Both replicate to each other. What is the final state? You must have an answer.

Conflict resolution strategies:

Last-write-wins with logical clocks: Each write has a timestamp (Hybrid Logical Clock — wall clock + logical component to handle clock skew). On conflict, the higher timestamp wins.

Simple. Problem: the losing write is silently discarded. Acceptable for some domains (last profile update wins), not for financial data.

Application-level conflict resolution: System detects a conflict and routes it to a human review queue. Both writes are preserved. A reviewer resolves. Expensive operationally. Correct for high-value conflicts (two different adjudication decisions for the same claim).

CRDT (Conflict-Free Replicated Data Types): Data structures that merge automatically without conflicts. Counters (always add, never set), sets (union merges), registers with defined semantics. Limited to specific data shapes but eliminates the conflict problem entirely for those shapes.

Domain-partitioned writes: Assign ownership of records to regions. US claims are owned by us-east. Indian claims owned by ap-south. A write to a record routes to the owning region. Other regions are read replicas. This is active-active for reads but active-passive per record for writes. Avoids conflicts entirely. Loses the "any region can serve any write" property but is far simpler to reason about.

The data replication layer:
AWS Aurora Global Database — synchronous replication within region, asynchronous cross-region with typically under 1-second lag. Promotion to a new primary in under 60 seconds on region failure.
Kafka MirrorMaker 2 — replicate Kafka topics across regions. Consumers in each region read from local brokers. Producers write to local brokers. MirrorMaker replicates bidirectionally.

What to say in the interview:
"Multi-region active-active is often over-engineered for the actual requirement. Before designing full active-active, I'd ask: is this a latency requirement or an availability requirement? Latency — active-active with domain partitioning. Availability — active-passive with fast failover (Aurora Global DB, sub-minute RTO) may be sufficient and dramatically simpler. I'd choose the simplest architecture that meets the actual SLA."

Q: How do you approach API design for a platform that will be consumed by 200 external partners?

External APIs are a contract. Every breaking change costs your partners engineering time. At 200 partners, a breaking change is a coordinated migration across 200 teams, many of whom are outside your organization and outside your influence. The cost is asymmetric — easy for you to change, very expensive collectively for your ecosystem.

Design for longevity from day one:
Version in the URL: `/v1/claims`. Not in headers (operationally harder to route). Not query params (gets dropped by proxies). URL versioning is visible, explicit, and easy to maintain in gateway routing rules.

Use additive changes only within a version: You can add new optional fields to a response. You can add new optional request parameters. You can add new endpoints. You cannot remove fields, rename fields, change field types, or change response structure without a new version.

Deprecation policy: Define it before you need it. "v1 will be supported for 24 months after v2 GA." Publish this. Put deprecation warnings in response headers (`Deprecation: true`, `Sunset: 2027-01-01`). Give partners enough time to migrate. Honor the policy — partners who built on your API based on your deprecation commitment will never trust you again if you break it early.

Schema design for external APIs:
Be conservative in what you accept (strict validation) and liberal in what you return (include more context than partners ask for — they'll use it later). Never return internal IDs that expose your implementation (auto-increment integer IDs reveal your data volume and are not portable across systems). Use UUIDs or opaque string identifiers.

Pagination, filtering, sorting — standardize across all endpoints:
Every list endpoint uses the same pagination contract. Cursor-based pagination scales better than offset. `{ "data": [...], "nextCursor": "opaque_string", "hasMore": true }`. Filtering: `GET /claims?status=SUBMITTED&submittedAfter=2026-01-01`. Consistent across every resource so partners learn one pattern.

Error responses — the most important consistency decision:
Every error from every endpoint uses the same structure. Partners write error handling once.

```
{
  "error": {
    "code": "CLAIM_VALIDATION_FAILED",
    "message": "Human readable description",
    "details": [{"field": "patientId", "issue": "required"}],
    "requestId": "trace-id-for-support",
    "documentationUrl":
```

```
  "https://docs.availity.com/errors/CLAIM_VALIDATION_FAILED"
    }
}
```

Machine-readable `code` for programmatic handling. Human-readable `message` for debugging. `requestId` for support conversations. `documentationUrl` so partners can self-serve.

The documentation is part of the API:

An API without documentation is incomplete. OpenAPI spec generated from your code (Springdoc), hosted as interactive docs (Swagger UI), versioned alongside the API. Examples for every endpoint — not just field definitions but complete request/response examples for common scenarios. A partner integration guide that walks through authentication, error handling, pagination, and webhooks end to end.

6/8/2026, 12:12:40 PM

is this much is enough

6/8/2026, 12:29:52 PM

Honest answer — **yes, for Senior. Almost for Principal.**

Here's the real breakdown:

What You Now Have — Genuinely Strong

- Distributed systems depth (Saga, Outbox, Consistent Hashing, CRDT, leader election)
- Production incident thinking
- Architecture decision frameworks (build vs buy, rewrite vs refactor, platform vs team)
- Organizational influence patterns
- Multi-region, observability platform, API design at scale
- End-to-end system designs with principal-level additions

This is **top 15% of senior engineer** knowledge coverage.

The Honest Gap — It's Not Knowledge Anymore

At this point your gap is **not content**. Reading more Q&A will give you diminishing returns.

The real gaps are:

1. You Haven't Practiced Saying It Out Loud

Reading a perfect answer and delivering it live under pressure are completely different skills. Interviewers interrupt. They challenge your choices. They ask "why not X instead?" You need to respond without your notes.

2. You Haven't Timed Yourself

A system design interview is 45 minutes. The distribution should be roughly:

```
0-5 min    → Requirements clarification
5-10 min   → Estimation + constraints
10-25 min  → High level design
25-35 min  → Deep dive on 1-2 components
35-45 min  → Failure modes + trade-offs
```

Most engineers spend 30 minutes on HLD and never get to failure modes — which is exactly where principal-level judgment is demonstrated.

3. You Haven't Practiced Being Challenged

Interviewers at principal level deliberately poke holes:

- *"Why Kafka here, wouldn't SQS be simpler?"*
- *"Your DB choice won't scale past 10M records"*
- *"What happens if the Saga orchestrator itself crashes?"*

The correct response is not to defend your design — it's to engage the tradeoff openly. Most candidates get defensive. That reads as insecurity, not seniority.

4. Estimation is Weak in Most Candidates

Back-of-envelope math separates principal candidates from senior candidates more than any other single skill. If you can't do this fluently:

```
1M claims/day
→ 12/sec average, 120/sec peak (10x)
→ 10KB payload = 120MB/sec peak ingest
→ 3.65TB/year storage
→ 10 events/claim = 10M audit events/day
→ Need 12 Kafka partitions minimum for peak throughput
→ Aurora write IOPS: 120 writes/sec × 3 (replication) = 360 IOPS
```

...you will be exposed at principal level even with perfect conceptual knowledge.

What's Still Worth Adding (Small Gaps)

These come up at principal level and you don't have deep coverage yet:

Topic	Why It Matters
Data modeling for scale	How you design schemas for 100M+ rows, denormalization decisions, materialized views
Cost optimization	Principal engineers think about infra cost — reserved instances, spot instances, S3 tiering, right-sizing
Security architecture	Zero trust, mTLS, secrets rotation, RBAC vs ABAC — at least one principal question touches this
Conway's Law applied	How org structure drives architecture — very common principal discussion
SLO/SLA/SLI definitions	How you define, measure, and negotiate reliability targets with business stakeholders

---

Realistic Assessment by Role

Senior Engineer Interview

You are ready. The content coverage is complete. Spend remaining time doing 3-5 full practice designs end-to-end, timed.

Principal Engineer Interview

You are 75-80% ready on content. The remaining 20-25% is:

- Live delivery under pressure
- Estimation fluency
- Handling challenges without getting defensive
- Demonstrating you think about org impact, not just technical correctness

---

What To Do From Here

If interview is within 2 weeks:

Stop reading new content. Do this instead:

Day 1-3: Practice these designs end-to-end out loud, timed at 45 min each:

- Design a claims processing system (your home turf)
- Design a notification system
- Design a rate limiter

Day 4-5: Practice estimation. For any system, derive: RPS, storage/year, bandwidth, number of servers needed.

Day 6-7: Practice being challenged. For every design decision you make, ask yourself *"why not the alternative?"* and answer it out loud.

Day 8+: Light review of weak areas only.

If interview is 1+ month away:

Cover the 5 small gaps in the table above, then follow the 2-week plan.

The One Thing That Predicts Success

Candidates who get principal offers don't just answer questions — they **drive the conversation**. They say:

"Before I jump into design, let me clarify two things that will significantly change the architecture..."

"I see two viable approaches here — let me walk through both and explain why I'd choose the first given these constraints..."

"The part I'm most uncertain about is X — here's how I'd validate that assumption..."

That posture — confident, open to tradeoffs, driving the room — is what principal interviews are actually testing.

Want to do a live practice design right now? Pick a system and I'll play interviewer — interruptions, challenges, and all.

6/8/2026, 12:30:26 PM

is it cover ?

TopicWhy It MattersData modeling for scaleHow you design schemas for 100M+ rows, denormalization decisions, materialized viewsCost optimizationPrincipal engineers think about infra cost — reserved instances, spot instances, S3 tiering, right-sizingSecurity architectureZero trust, mTLS, secrets rotation, RBAC vs ABAC — at least one principal question touches thisConway's Law appliedHow org structure drives architecture — very common principal discussionSLO/SLA/SLI definitionsHow you define, measure, and negotiate reliability targets with business stakeholders

6/8/2026, 12:32:04 PM

Not yet. Let me cover all 5 now.

Principal Engineer — The 5 Missing Topics

1. Data Modeling for Scale

The Core Problem

A schema that works perfectly at 1M rows starts breaking at 100M rows. Not because SQL is wrong — because the design assumptions change. At scale, every JOIN is expensive, every full-table scan is catastrophic, and every index adds write overhead.

The shift in thinking:
At small scale → normalize everything, avoid duplication, let the DB do the work via JOINs.
At large scale → denormalize deliberately, duplicate data to avoid JOINs, move computation to write time not read time.

Normalization vs Denormalization at Scale

Normalized (correct at small scale):

```
claims       (claimId, patientId, payerId, status, amount)
patients     (patientId, name, dob, memberId)
payers       (payerId, name, contractType, npiNumber)

-- Query for claim list view:
SELECT c.claimId, c.status, c.amount,
       p.name as patientName,
       py.name as payerName
FROM claims c
JOIN patients p ON c.patientId = p.patientId
JOIN payers py  ON c.payerId = py.payerId
WHERE c.tenantId = ? AND c.status = 'SUBMITTED'
ORDER BY c.createdAt DESC
LIMIT 20;
```

At 100M claims, 10M patients, 5000 payers — this JOIN scans a massive index, executes two lookups per row, and under concurrent load kills your DB.

Denormalized (correct at large scale):

```
claims (
  claimId, patientId, payerId, status, amount,
  -- denormalized read fields:
  patientName, patientDob, patientMemberId,
  payerName, payerContractType
)
```

The claim list query becomes:

```
SELECT claimId, status, amount, patientName, payerName
FROM claims
WHERE tenantId = ? AND status = 'SUBMITTED'
ORDER BY createdAt DESC
LIMIT 20;
-- No JOINs. Single table scan on (tenantId, status, createdAt)
index.
```

The tradeoff you accept: If a patient's name changes, you have to update every claim row for that patient. This is acceptable in healthcare — patient name on a submitted claim is the name at submission time, not the current name. The business rule validates the denormalization.

Always ask: Does the denormalized copy need to be the current value or the value-at-time-of-event? For audit and financial records, value-at-event is usually correct anyway.

Index Design at Scale

Indexes are not free. Every index adds overhead to every INSERT, UPDATE, DELETE. At 120 writes/second, a table with 12 indexes is doing 1440 index updates per second. Design indexes for your actual query patterns.

Composite index column order matters:

```
-- Query: WHERE tenantId = ? AND status = ? ORDER BY createdAt DESC
-- Correct index:
CREATE INDEX idx_claims_tenant_status_created
ON claims (tenantId, status, createdAt DESC);

-- Wrong index (status first — tenantId not used for range scan):
CREATE INDEX idx_claims_status_tenant
ON claims (status, tenantId, createdAt DESC);
```

Rule: most selective filter first, then other filters, then sort column last.

Covering index — avoid table lookups entirely:

```
-- Query only needs these 4 columns:
SELECT claimId, status, amount, createdAt
FROM claims
WHERE tenantId = ? AND status = 'SUBMITTED'

-- Covering index includes all selected columns:
CREATE INDEX idx_claims_covering
ON claims (tenantId, status, createdAt DESC)
```

```
INCLUDE (claimId, amount);
-- DB serves query entirely from index — never touches the table
```

Partial indexes (PostgreSQL):

```
-- Only index active claims — not the 90% that are SETTLED/ARCHIVED
CREATE INDEX idx_claims_active
ON claims (tenantId, createdAt DESC)
WHERE status IN ('SUBMITTED', 'PENDING', 'IN_REVIEW');
-- 10x smaller index, 10x faster for active claim queries
```

Materialized Views

A materialized view stores the result of a complex query physically on disk. Instead of computing an expensive aggregation at query time, you compute it at write time and read the precomputed result.

Use case — claims summary dashboard:

```
-- Without materialized view: runs on every dashboard load
SELECT tenantId,
       COUNT(*) as totalClaims,
       SUM(amount) as totalAmount,
       COUNT(CASE WHEN status='APPROVED' THEN 1 END) as approved,
       COUNT(CASE WHEN status='DENIED' THEN 1 END) as denied
FROM claims
WHERE createdAt >= NOW() - INTERVAL 30 DAY
GROUP BY tenantId;
-- At 100M rows: full table scan, 5-10 seconds

-- With materialized view: precomputed
CREATE MATERIALIZED VIEW claims_summary_30d AS
SELECT tenantId,
       COUNT(*) as totalClaims,
       SUM(amount) as totalAmount,
       COUNT(CASE WHEN status='APPROVED' THEN 1 END) as approved,
```

```
       COUNT(CASE WHEN status='DENIED' THEN 1 END) as denied
FROM claims
WHERE createdAt >= NOW() - INTERVAL 30 DAY
GROUP BY tenantId;

-- Dashboard query:
SELECT * FROM claims_summary_30d WHERE tenantId = ?;
-- Sub-millisecond
```

Refresh strategies:

- Full refresh (simple, expensive): `REFRESH MATERIALIZED VIEW claims_summary_30d` — recomputes everything. Run as a scheduled job (nightly, hourly).
- Incremental refresh: Only recompute rows affected by recent changes. More complex, supported natively in some DBs (Oracle, Snowflake), requires custom triggers in PostgreSQL/MySQL.
- Event-driven refresh: Kafka consumer listens for claim state change events, updates the materialized view incrementally. Most responsive, most complex.

In your stack: Aurora MySQL doesn't have native materialized views. Implement via: a `claims_summary` table maintained by a Kafka consumer that processes claim events, or a Redis hash updated on every claim state change, or a scheduled job that runs the aggregation and upserts into a summary table.

Schema Evolution at Scale

Changing a schema on a 100M row table is an operational event, not a code change.

The problem: `ALTER TABLE claims ADD COLUMN authorizationCode VARCHAR(50)` on a 100M row table in MySQL locks the table for 20-30 minutes. Your service is down.

The solution — online schema changes:

- **pt-online-schema-change (Percona):** Creates a new table with the new schema, copies data in batches, uses triggers to sync changes during copy, atomically renames. Zero downtime.

- **gh-ost (GitHub):** Similar approach but uses binlog instead of triggers. More reliable under heavy write load. GitHub open-sourced it — widely used in production MySQL systems.
- **Aurora's instant DDL:** `ADD COLUMN` for nullable columns with no default is instant in Aurora MySQL 8.0 — metadata-only change. Use this when possible.

Expand-contract pattern for application-level safety:

```
Phase 1 (expand):   Add new column as nullable. Deploy code that
writes both
                    old and new columns. Reads from old column.
Phase 2 (migrate):  Backfill new column for existing rows in
batches.
Phase 3 (switch):   Deploy code that reads from new column. Writes
only new column.
Phase 4 (contract): Drop old column (now safe — no code reads it).
```

Never skip phases. Each phase is a separate deployment. You can pause between phases if something goes wrong.

Time-Series Data Modeling

Claims data, audit logs, and events are time-series by nature. Model them accordingly.

Partition by time (as covered in partitioning notes) — range partition on `createdAt` by month or year. Queries are almost always time-bounded. Old partitions can be archived.

Write-optimized append pattern:
Never UPDATE audit rows. Only INSERT. The current state is derived from the latest event, not stored directly. This is event sourcing at the table level. Append-only tables have no UPDATE lock contention — massive write throughput improvement.

Retention and archival:
Hot data (last 90 days) → Aurora MySQL, full indexes, fast queries.
Warm data (90 days - 2 years) → Aurora with fewer indexes, or cheaper RDS instance.
Cold data (2+ years) → S3 in Parquet format, queryable via Athena for compliance audits.

```
-- Athena query on archived S3 data (no DB needed):
SELECT claimId, status, amount
FROM s3_claims_archive
WHERE year = 2023 AND tenantId = 'availity-001'
AND status = 'DENIED';
```

---

2. Cost Optimization

Why Principal Engineers Must Think About Cost

A system that costs $2M/month to run is not a good system even if it's technically elegant. At principal level you're expected to make architecture decisions that balance capability, reliability, and cost. "We'll just throw more machines at it" is a junior answer.

AWS Compute Cost Optimization

On-Demand vs Reserved vs Spot:

Type	Cost	Use case
On-Demand	100% (baseline)	Unpredictable workloads, new services
Reserved (1-year)	~40% cheaper	Stable baseline load you know you'll need
Reserved (3-year)	~60% cheaper	Core infrastructure that won't change
Savings Plans	~40-60% cheaper	More flexible than Reserved — commits to $ spend not instance type
Spot	~70-90% cheaper	Fault-tolerant, interruptible workloads

Practical strategy:
Run baseline load (60-70% of normal capacity) on Reserved instances — you know you need these always.
Run variable load on On-Demand or Savings Plans.

Run batch jobs, Kafka consumers for non-critical topics, ML training on Spot — they can handle interruption with proper checkpoint/resume logic.

Right-sizing:
The most common waste is over-provisioned instances. An `m5.4xlarge` running at 15% CPU average is paying for 85% idle capacity.
Use AWS Compute Optimizer — it analyzes CloudWatch metrics and recommends the right instance type.
In Kubernetes: set resource requests accurately. If every pod requests 2CPU but uses 0.3CPU, your cluster is 6x over-provisioned. HPA and Cluster Autoscaler work on requests — accurate requests = accurate scaling.

```
# Right-sized Kubernetes resource spec
resources:
  requests:
    memory: "512Mi"    # what it actually uses at P50
    cpu: "250m"        # what it actually uses at P50
  limits:
    memory: "1Gi"      # headroom for spikes
    cpu: "1000m"       # throttle ceiling
```

Database Cost Optimization

Aurora costs: Read replicas are expensive — each replica is ~60-70% of primary cost. Don't add replicas reflexively. Add them when you have measurable read load that the primary can't handle.

RDS vs Aurora:
Aurora is 3-5x more expensive than RDS MySQL for the same instance size. For low-traffic services, RDS MySQL is sufficient. Reserve Aurora for services that need its specific capabilities: instant failover (sub-30s), Aurora Serverless for variable load, Global Database for multi-region.

Aurora Serverless v2:
Scales from 0.5 ACUs to 128 ACUs in seconds. Pay per ACU-hour only when active. Perfect for: dev/staging environments (scale to zero at night), variable workloads with unpredictable spikes, new services where traffic is unknown.

Not good for: sustained high-throughput workloads — provisioned Aurora is cheaper at consistent high load.

Read query cost reduction:
Move reporting and analytics queries off Aurora entirely. These are expensive (long-running, full scans) and competing with transactional queries. Route them to: Elasticsearch (for search/aggregation), Redshift (data warehouse), or Athena on S3 (for historical data). Your Aurora primary handles only transactional workload — everything else goes elsewhere.

Storage Cost Optimization

S3 storage tiers — use lifecycle policies:

```
S3 Standard          → Hot data, frequent access
($0.023/GB/month)
S3 Standard-IA       → Infrequent access
($0.0125/GB/month)
S3 Glacier Instant   → Archive, millisecond access
($0.004/GB/month)
S3 Glacier Flexible  → Archive, hours to access
($0.0036/GB/month)
S3 Deep Archive      → Long-term, 12hr retrieval
($0.00099/GB/month)
```

Lifecycle policy for claims data:

```
{
  "Rules": [{
    "Transitions": [
      { "Days": 90,  "StorageClass": "STANDARD_IA" },
      { "Days": 365, "StorageClass": "GLACIER_INSTANT_RETRIEVAL" },
      { "Days": 2555, "StorageClass": "DEEP_ARCHIVE" }
    ]
  }]
}
```

At 3.65TB/year for claims data, this lifecycle saves ~70% on storage costs vs keeping everything in S3 Standard.

Compress everything on S3:
Parquet format with Snappy compression reduces file sizes by 5-10x compared to raw JSON. A 100GB JSON export becomes 10-20GB in Parquet. Storage cost and Athena query cost (billed per TB scanned) both drop proportionally.

Kafka Cost Optimization

Kafka cost is dominated by: broker compute, storage (retention × throughput), and data transfer.

Retention tuning:
Don't retain data in Kafka longer than consumers need it. If your slowest consumer catches up within 24 hours, 7-day retention is paying for 6 days of unnecessary storage. Set retention based on measured consumer recovery time + safety margin.

Tiered storage (Confluent/MSK):
Move old Kafka segments to S3 automatically. Brokers only keep recent data in local storage — historical data fetched from S3 when needed (rare). Dramatically reduces broker disk size and cost.

Compression:
Enable producer-side compression (`compression.type=lz4` or `snappy`). Reduces network transfer and broker storage. LZ4 is fast with good compression ratio — good default for most payloads. GZIP gives better compression but higher CPU overhead.

Cost Monitoring

You can't optimize what you don't measure.

AWS Cost Explorer: Tag every resource with `service`, `team`, `environment`. Cost Explorer shows you spend per tag. You can tell the claims team exactly what their service costs per month.

Kubecost: Shows Kubernetes cost per namespace, per deployment, per pod. Tells you exactly which service is consuming what in your cluster. Essential for chargeback in multi-team environments.

Budget alerts: Set AWS Budgets alerts at 80% and 100% of monthly budget per service. Get a Slack notification before you get a bill surprise.

---

3. Security Architecture

Zero Trust Architecture

Traditional security model: trust everything inside the network perimeter. Once inside the VPC, services can talk to each other freely.

Zero trust: **never trust, always verify**. Every request — even internal service-to-service — must be authenticated and authorized. The network perimeter is not a security boundary.

Why this matters: if one service is compromised in a perimeter-trust model, the attacker has lateral movement access to every other service. In zero trust, compromising one service gives access only to what that service is explicitly authorized to reach.

Implementation layers:

Layer 1 — mTLS between services (mutual TLS):
Both client and server present certificates. The server verifies the client is who it claims to be — not just any service inside the VPC. Implemented via service mesh (Istio, Linkerd) — the sidecar proxy handles mTLS transparently, no application code change needed.

```
# Istio PeerAuthentication — require mTLS for all services in
namespace
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: claims-platform
spec:
  mtls:
    mode: STRICT  # reject any non-mTLS traffic
```

Layer 2 — Service-to-service authorization (AuthorizationPolicy):
Even with mTLS proving identity, restrict which services can call which endpoints.

```
# Only eligibility-service can call claims-service /internal/claims
endpoint
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: claims-service-policy
spec:
  selector:
    matchLabels:
      app: claims-service
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/default/sa/eligibility-
service"]
    to:
    - operation:
        paths: ["/internal/claims/*"]
```

Layer 3 — JWT validation at API Gateway:
External requests carry JWT tokens. API Gateway (Tyk) validates signature, expiry, and claims before forwarding. Services should also validate tokens — gateway compromise shouldn't give access to services directly.

Secrets Management

Never store secrets in code, config files, or environment variables in plain text. This is a HIPAA violation risk and a security fundamental.

AWS Secrets Manager:

```
// Spring Boot — fetch secret at startup, not hardcoded
@Value("${DB_PASSWORD}")  // ❌ wrong — sourced from env var or
config
String dbPassword;
```

```
// ✅ correct — fetch from Secrets Manager via AWS SDK or Spring Cloud AWS
@Bean
public DataSource dataSource(SecretsManagerClient secretsClient) {
    GetSecretValueResponse secret = secretsClient.getSecretValue(
        GetSecretValueRequest.builder()
            .secretId("prod/claims-service/db-password")
            .build()
    );
    // use secret.secretString() to configure DataSource
}
```

Secret rotation:
AWS Secrets Manager rotates secrets automatically on a schedule. Your application must handle secret rotation without restart — use a connection pool that validates connections on borrow (HikariCP `connection-test-query`) so stale connections with old passwords are detected and replaced.

Kubernetes secrets — the gotcha:
Kubernetes Secrets are base64-encoded, not encrypted by default. Anyone with kubectl access can decode them. Use External Secrets Operator to sync from AWS Secrets Manager into Kubernetes Secrets — secrets never live in Git or etcd in plaintext.

RBAC vs ABAC

RBAC (Role-Based Access Control):
Permissions are assigned to roles. Users are assigned to roles. Simple, well-understood, easy to audit.

```
Roles:
  CLAIM_SUBMITTER  → can POST /claims
  CLAIM_REVIEWER   → can GET /claims, PUT /claims/{id}/status
  CLAIM_ADMIN      → all of the above + DELETE

User Alice → role CLAIM_REVIEWER
User Bob   → role CLAIM_SUBMITTER
```

Good for: most enterprise applications where job function maps cleanly to access patterns. Easy to implement with Spring Security.

ABAC (Attribute-Based Access Control):
Permissions based on attributes of the user, the resource, and the environment. More expressive, more complex.

```
Policy: Allow access if
  user.tenantId == resource.tenantId
  AND user.role == 'CLAIM_REVIEWER'
  AND resource.status != 'ARCHIVED'
  AND request.time is BUSINESS_HOURS
```

Good for: multi-tenant systems (a reviewer should only see their own tenant's claims), data-level access control, compliance-heavy domains. HIPAA minimum necessary access principle requires ABAC — a nurse should see patient records for their department, not all patients.

Spring Security implementation:

```
// RBAC
@PreAuthorize("hasRole('CLAIM_REVIEWER')")
public Claim getClaim(String claimId) { ... }

// ABAC — method security with custom expression
@PreAuthorize("@claimAccessPolicy.canAccess(authentication,
#claimId)")
public Claim getClaim(String claimId) { ... }

// Policy bean evaluates tenant isolation + role + claim state
@Component
public class ClaimAccessPolicy {
    public boolean canAccess(Authentication auth, String claimId) {
        UserPrincipal user = (UserPrincipal) auth.getPrincipal();
        Claim claim = claimRepository.findById(claimId);
        return claim.getTenantId().equals(user.getTenantId())
            && user.hasRole("CLAIM_REVIEWER");
```

```
    }
}
```

In healthcare: You need both. RBAC for coarse-grained role enforcement (only doctors can prescribe). ABAC for fine-grained data access (this doctor can only see their assigned patients). HIPAA minimum necessary standard is fundamentally an ABAC requirement.

Security at Principal Level — The Big Picture

Principal engineers own security architecture decisions, not just implementation. The questions you should be able to answer:

Threat modeling: For a new system, what are the assets (PHI, payment data), who are the threats (external attackers, insider threats, compromised services), what are the attack vectors (injection, broken auth, insecure direct object reference), and what controls mitigate each?

Defense in depth: Security controls at every layer — network (VPC, security groups, WAF), transport (TLS, mTLS), application (auth, input validation), data (encryption at rest, field-level encryption for PHI), audit (immutable access logs). No single layer is sufficient.

Encryption at rest: Aurora encryption with AWS KMS. S3 SSE-KMS. Redis encryption at rest. Kafka encryption at rest (MSK with KMS). Key rotation policy. For HIPAA PHI — field-level encryption for sensitive fields (SSN, diagnosis codes) using application-layer encryption so even DB admins can't read raw PHI.

---

4. Conway's Law Applied

What Conway's Law Actually Says

"Any organization that designs a system will produce a design whose structure is a mirror image of the organization's communication structure."

Melvin Conway, 1967. Still the most practically useful law in software architecture.

This isn't just an observation — it's a constraint. If you have 4 teams, you will end up with 4 subsystems whether you intend to or not, because each team optimizes for their own area and the interfaces between teams become the interfaces between systems.

Why It Matters for Principal Engineers

If you design an architecture and then try to staff it with a team structure that doesn't match, one of two things happens: the architecture drifts toward the team structure (the teams win), or the teams are miserable because they're constantly stepping on each other across poorly-matched boundaries (the architecture appears to win but the velocity suffers).

Principal engineers design the team structure and the architecture together. You cannot do one without considering the other.

Practical Examples

Bad alignment — Conway's Law working against you:

You design a clean microservices architecture: Claims Service, Eligibility Service, Adjudication Service.

But your team structure is: Frontend Team, Backend Team, Data Team.

The Backend Team owns all three services. Every feature requires coordination within one team but the services are now artificially separated. The team will gradually merge the services back (or add shared libraries that create tight coupling) because the separation creates friction with no benefit.

Good alignment — Conway's Law working for you:

Team structure mirrors service boundaries:

- Claims Team → owns Claims Service + Claims DB
- Eligibility Team → owns Eligibility Service + Eligibility DB
- Adjudication Team → owns Adjudication Service + Adjudication DB

Each team deploys independently. API contracts are the interface between teams. Team autonomy and service autonomy are the same thing. Conway's Law produces the architecture you wanted.

Inverse Conway Maneuver

Deliberately restructure your teams to produce the architecture you want.

If you want to move from a monolith to microservices, restructure teams first. Create small, product-aligned teams with end-to-end ownership. The architecture will follow. Trying to

decompose the monolith with a team structure that doesn't match the target architecture will fail — the team boundaries will keep pulling the architecture back toward the monolith.

How to apply it:
Define the target architecture (what services do you want?). Define the team topology that would naturally produce that architecture. Restructure teams to match. Let the architecture emerge from team autonomy. The technical decomposition work is easier once the team structure is right.

Team Topologies (Matthew Skelton & Manuel Pais)

The modern framework for thinking about team structure and architecture together. Four team types:

Stream-aligned teams: Own a product/service end to end — from frontend to DB to deployment. Autonomous, fast-moving. This is your Claims Team, Eligibility Team, etc. Most teams should be stream-aligned.

Platform teams: Build and maintain internal platforms that stream-aligned teams use — Kubernetes platform, CI/CD pipelines, observability stack, shared auth library. Reduce cognitive load for stream-aligned teams by handling the "how do I run this in production" problem.

Enabling teams: Temporary teams that help stream-aligned teams adopt new practices — "we're here for 3 months to help you move to event-driven architecture, then you own it." Not permanent. Not a bottleneck.

Complicated subsystem teams: Own components that require deep specialist knowledge — ML models, cryptography, real-time pricing engines. Small teams, very specialized. Stream-aligned teams consume their output via well-defined APIs.

The interaction modes:

- Collaboration: two teams work closely together (temporary, for exploration)
- X-as-a-Service: platform team provides something, stream-aligned team consumes it (minimal interaction)
- Facilitating: enabling team coaches stream-aligned team (temporary)

For your interview: "I'd organize the engineering org as stream-aligned teams per business domain — Claims, Eligibility, Provider Management — each with full-stack

ownership. A platform team handles Kubernetes, CI/CD, and observability. This team structure directly produces the service architecture we want and avoids the coordination overhead of functional teams."

Communication Overhead and Architecture Complexity

Brook's Law: adding people to a late project makes it later. The reason is communication overhead — N people have N×(N-1)/2 communication paths.

The same applies to services. N services have N×(N-1)/2 potential integration paths. Every integration path is a potential failure point, a security surface, a versioning concern.

The practical implication: Don't decompose services beyond what your team structure can naturally support. A 5-person team owning 15 microservices has the same communication overhead problem as 15 people on one project — too many moving parts, too many interfaces to maintain. Right-size your service granularity to your team size.

Rule of thumb: one team should own 1-4 services. If a team owns more, services are probably too granular. If multiple teams own one service, you have a coordination problem.

---

5. SLO / SLA / SLI Definitions

The Definitions

SLI (Service Level Indicator): A specific metric that measures one aspect of service health. It's the raw measurement.

Examples:

- Request success rate: `successful_requests / total_requests`
- Latency: P99 response time in milliseconds
- Availability: `uptime_minutes / total_minutes`
- Throughput: requests processed per second
- Error rate: `5xx_responses / total_responses`

SLO (Service Level Objective): A target value for an SLI. Internal agreement — what the engineering team commits to.

Examples:

- Success rate SLO: 99.9% of requests succeed
- Latency SLO: P99 < 500ms
- Availability SLO: 99.95% uptime per month

SLA (Service Level Agreement): A contract with external parties (customers, partners) that includes consequences for violation — usually financial penalties or service credits.

SLA is always weaker than SLO. You promise customers 99.9% (SLA). You target 99.95% internally (SLO). The gap is your error budget buffer — if you hit your SLO, you're not at risk of violating your SLA.

```
SLI → what you measure
SLO → what you target internally
SLA → what you promise externally (with consequences)

SLA ≤ SLO (SLA is always easier to meet than SLO)
```

Error Budget

Error budget = 100% - SLO target. It's the amount of unreliability you're allowed before violating your SLO.

```
SLO: 99.9% availability per month
Month = 30 days = 43,200 minutes
Error budget = 0.1% × 43,200 = 43.2 minutes of downtime allowed

SLO: 99.95% availability
Error budget = 0.05% × 43,200 = 21.6 minutes allowed

SLO: 99.99% availability ("four nines")
Error budget = 0.01% × 43,200 = 4.32 minutes allowed
```

Error budget as a management tool:
This is the key insight from Google's SRE book. Error budget converts reliability from a vague aspiration into a quantified resource that engineering and product teams share.

If you have 40 minutes of error budget remaining this month and a risky deployment could consume 20 minutes if it goes wrong — that's a conversation with real numbers. If you've already consumed 35 of your 43 minutes, you're in error budget freeze — no risky deployments until next month.

If you have plenty of error budget remaining, you can move fast. If it's depleted, you slow down and invest in reliability. This creates the right incentive structure — teams aren't punished for incidents as long as they're within budget, and they're not rewarded for over-reliability that's slowing feature delivery.

Designing Good SLOs

Make SLOs user-focused:
Bad SLO: CPU < 80%. This is a resource metric, not a user experience metric. CPU at 79% is meaningless to a user if their requests are slow.
Good SLO: P99 latency < 500ms for claim submission, measured at the API Gateway, over a 30-day rolling window.

The four golden signals as SLO candidates:

- Latency (P99, not average — average hides tail latency)
- Error rate (% of requests returning 5xx or business errors)
- Availability (% of time the service is serving requests successfully)
- Throughput (only if business-critical to maintain minimum processing rate)

Multi-window alerting:
Alert when you're burning error budget too fast — not just when you've exhausted it.

```
Fast burn (1-hour window):   consuming 2% of monthly budget in 1 hour → page immediately
Slow burn (6-hour window):   consuming 5% of monthly budget in 6 hours → ticket
Exhaustion alert:            <10% error budget remaining → deployment freeze
```

Practical SLO for Claims Service:

```yaml
slos:
  claim_submission_availability:
    sli: successful_submissions / total_submissions
    target: 99.9%
    window: 30_days

  claim_submission_latency:
    sli: P99 latency of POST /claims
    target: < 500ms
    window: 30_days

  claim_processing_throughput:
    sli: claims_processed_per_hour
    target: > 40000  # 1M/day minimum
    window: 24_hours
```

Negotiating SLOs with Business Stakeholders

This is the principal-level skill — translating technical SLOs into business conversations.

The cost of reliability is exponential:

Availability	Downtime/month	Relative cost
99%	7.3 hours	1x
99.9%	43 minutes	~3x
99.99%	4.3 minutes	~10x
99.999%	26 seconds	~30x+

Moving from 99.9% to 99.99% is not a 0.09% improvement — it's 10x more engineering investment in redundancy, testing, deployment automation, and oncall burden. This cost must be justified by business value.

The conversation to have:
"Our claims submission API is currently at 99.9% — about 43 minutes of downtime per

month. Getting to 99.99% requires multi-region active-active, which is a 6-month engineering project and ongoing operational cost. What is the business cost of 43 minutes of downtime per month? If it's less than the engineering investment to prevent it, 99.9% is the right target."

This is principal-level thinking — connecting reliability targets to business outcomes, not chasing four nines because it sounds impressive.

Now You Have Full Coverage

Topic	Status
System design fundamentals	✅ Complete
Distributed systems patterns	✅ Complete
Performance & scalability	✅ Complete
DB partitioning	✅ Complete
Principal Q&A (architecture decisions)	✅ Complete
Data modeling for scale	✅ Just covered
Cost optimization	✅ Just covered
Security architecture	✅ Just covered
Conway's Law + team topologies	✅ Just covered
SLO/SLA/SLI	✅ Just covered

The content gap is now closed. The only remaining gap is practice — live delivery under pressure.

Pick a system and do a timed 45-minute end-to-end design. That's the last step.

6/8/2026, 12:35:02 PM